



Chat-First Data Analytics

Ulrich Matter

Bern University of Applied Sciences
University of St. Gallen

Abstract

Using natural language to drive data analysis is an old idea, but large language models have made it suddenly ubiquitous. A widely observed and rather concerning workflow has users upload a dataset to ChatGPT and ask for an analysis; the model generates and executes code on a remote virtual machine, producing results that are difficult to verify, impossible to reproduce, and incompatible with data privacy requirements. This paper develops *chat-first data analytics*, an alternative architecture built on three principles: separate interpretation from computation, pre-validate methods rather than generating code, and transmit only metadata, not data. The language model interprets intent and selects tools; pre-implemented, validated algorithms produce results. **prompt2analytics** implements this approach as an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The library covers regression, panel data, instrumental variables, difference-in-differences, discrete choice, time series, and over 200 additional methods—all validated against established R packages. Evaluation shows that tool selection works well (90–100% on curated prompts). Local deployment via Ollama enables complete data isolation on consumer hardware.

Keywords: chat-first analytics, natural language interface, econometrics, Rust, MCP, local LLM, data privacy.

1. Introduction

The idea of talking to a computer to analyze data is not new. Natural language interfaces to databases date to the 1970s (Codd 1974), and every generation of statistical software has tried to lower the barrier between analytical intent and software commands. What *is* new is that the idea suddenly works well enough to be everywhere. Since late 2022, millions of users have been typing requests like “regress wages on education, controlling for experience, with clustered standard errors” into ChatGPT and getting results back. The conversational

interface to data analysis has gone from research curiosity to mass-market product in under two years.

The results are concerning. In the most common workflow, a user uploads a CSV file to ChatGPT and types “run a regression.” The model generates a Python script and executes it on a remote virtual machine—the user’s data now resides on a third-party server they do not control. Even when users copy generated code back to their own machine, the problems persist: generated code may compute the wrong quantity without raising an error (Ludwig, Mullainathan, and Rambachan 2024), and the same prompt yields different code as models update—making results irreproducible. Section 2 examines these failure modes in detail.

The core idea—natural language as interface—is sound. The architecture is not. These problems are not inherent to conversational data analysis; they arise from a specific design choice: delegating computation to generated code. This paper develops an alternative called *chat-first data analytics*. The idea can be stated in three principles:

1. **Interpretation and computation are separated.** The language model selects methods and extracts parameters from natural language; pre-implemented, validated algorithms perform the actual computation.
2. **Methods are pre-validated, not generated.** Correctness is established once, through systematic validation against reference implementations, rather than hoped for anew with each request.
3. **Data stays local; only structure travels.** The language model receives column names, data types, and summary statistics—not the data itself. For complete isolation, the language model can run locally too.

These principles address each concern directly. Correctness follows from using validated implementations rather than generated code. Privacy is protected by keeping data on the user’s machine. Reproducibility is achieved because the same tool invocation produces identical results—the stochastic element of the language model affects only interpretation, not computation.

This paper presents **prompt2analytics**, an implementation of chat-first data analytics. The software includes an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The implementation covers regression analysis with robust standard errors, panel data models, instrumental variables, difference-in-differences, discrete choice, time series, and over 200 additional methods—all validated against established R packages.

The remainder of the paper is organized as follows. Section 2 develops the concept of chat-first analytics, examining the failure modes of current approaches and situating the three principles within existing work. Section 3 describes the software implementation. Section 4 demonstrates the system through worked examples including realistic failure cases. Section 5 evaluates accuracy across tool selection, multi-turn conversations, and parameter extraction. Section 6 presents performance benchmarks and local deployment configurations. Section 7 discusses limitations and future directions.

1.1. Software availability

prompt2analytics is available as open-source software under the MIT license at:

<https://github.com/umatter/prompt2analytics>

Supplementary materials include the 87 evaluation test cases, benchmark scripts for both R and Rust, and validation test suites. These are located in the repository’s `paper/` directory. Appendix G provides reproduction instructions.

2. Defining chat-first analytics

The introduction identified three concerns with using language models for data analysis: correctness, privacy, and reproducibility. This section examines these failure modes more carefully, then develops the three principles that define chat-first analytics.

2.1. How LLM-assisted analysis goes wrong

The appeal of typing “regress wages on education with robust standard errors” into ChatGPT is obvious. The problems are less visible. They fall into four categories.

Silent computational errors. LLM-generated code often runs without error but computes the wrong quantity. Requesting “clustered standard errors” might yield heteroskedasticity-robust errors instead. A panel model might omit entity fixed effects. A difference-in-differences specification might use the wrong interaction term. Ludwig *et al.* (2024) document these risks systematically: the code executes, returns plausible-looking output, and the user has no way to detect the error without the expertise that would have let them write the code themselves.

Privacy violations. Code-generation systems require the data. When a user asks ChatGPT to analyze a dataset, the data is uploaded to a third-party server with unclear retention policies. For survey responses, health records, personnel data, or any dataset subject to privacy regulations, this may be unacceptable. Even when providers promise not to retain data, the transmission itself may violate institutional or regulatory requirements.

Reproducibility failures. The same prompt sent to the same model on different days may produce different code. Models are updated, fine-tuned, and retrained. The generated Python script that worked in January may not be what the model produces in March. Conversational sessions are ephemeral—unless the user manually saves the generated code, the analysis cannot be reproduced. This contradicts a basic requirement of scientific work.

Unauditable reasoning. When a system generates 30 lines of Python to answer a statistical question, understanding what was computed requires reading and auditing 30 lines of code. The user who could not write the code cannot audit it either. This problem extends beyond research: enterprise analytics vendors have retreated from “chat with your data” products after discovering that users cannot verify whether identical questions yield consistent answers across data sources (Korinek 2025).

These four problems are distinct but related. Silent errors arise because the language model controls both *what* to compute and *how* to compute it. Privacy violations arise because computation and data must be co-located on the provider’s servers. Reproducibility failures arise because the generated code is a function of the model’s current state. Auditability failures arise because the computational logic is opaque—buried in generated code rather than invoked from a known, inspectable implementation.

2.2. Three principles

Chat-first data analytics addresses these problems through three principles. The first two are definitional; the third is a design choice that we consider essential.

Principle 1: Separate interpretation from computation

The language model decides *what* to compute; validated software decides *how*. When a user asks to “estimate the effect of education on wages, controlling for experience, with robust standard errors,” the language model’s job is to determine that this requires OLS with HC1 standard errors, identify the column names, and generate a structured tool call. The actual matrix algebra—forming $(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$, computing the HC1 sandwich estimator—is performed by pre-implemented code that the language model cannot modify.

This division of labor plays to each component’s strengths. Language models are good at understanding intent, resolving ambiguity, and mapping natural language to structured specifications. They are unreliable at generating correct numerical code. Separation eliminates the failure mode where the model produces code that runs but computes the wrong quantity, because the model never produces code at all.

The idea has a precedent in text-to-SQL systems (Yu, Zhang, Yang, Yasunaga, Wang, Li, Ma, Li, Yao, Roman, Zhang, and Radev 2018; Li, Luo, Chai, Li, and Tang 2024), where a language model translates natural language into structured database queries executed by a SQL engine. We extend this pattern from data retrieval to statistical computation: from text-to-SQL to text-to-statistics.

Principle 2: Pre-validate methods, not generated code

Correctness is established once, through systematic validation against reference implementations, rather than hoped for anew with each request. When **prompt2analytics** runs OLS, it uses the same `Rust` function every time—a function validated against R’s `lm()` on the Longley dataset to 10^{-8} precision (Section 5). The user need not audit the implementation on each invocation, any more than they audit R’s source code when calling `lm()`.

This principle also solves the reproducibility problem. The same method invocation with the same data produces identical results, regardless of when it is run or which language model is used. The stochastic element of the language model affects only interpretation (which tool to call, how to explain the output), never computation. Two users who phrase the same analysis differently may get different tool calls—but once the tool is selected and parameters specified, the result is deterministic.

Principle 3: Keep data local; transmit only structure

The language model receives column names, data types, and summary statistics—not the data itself. A dataset of 100,000 salary records is represented to the model as metadata: “columns: employee_id (int), salary (float, range 28000–185000), department (string, 12 unique values), years_experience (float, range 0.5–35.2).” Statistical computations execute on the user’s machine; only aggregated results (coefficients, standard errors, test statistics) pass through the language model.

This is not definitionally required for chat-first analytics—one could build such a system with cloud computation. But we consider it essential for the use cases that motivate this work.

Researchers analyzing survey data, firms analyzing personnel records, and analysts working under privacy regulations need architectures where data isolation is guaranteed by design, not promised by policy.

For complete isolation, the language model itself can run locally. Open-source models deployed via Ollama (Ollama Inc. 2024) eliminate all external network communication. Our evaluation (Section 5) shows that even 3-billion-parameter models achieve over 90% tool selection accuracy, making fully local deployment practical on consumer hardware.

2.3. What the principles buy you

Table 1 maps principles to the failure modes they address. The pattern is clear: separating interpretation from computation prevents silent errors, pre-validation ensures reproducibility, and data locality protects privacy. Auditability improves on all three fronts—the user inspects a tool call (“OLS, $y = \text{salary}$, $x = \text{education experience}$, $\text{robust} = \text{HC1}$ ”) rather than 30 lines of generated code.

Table 1: Principles mapped to failure modes

Failure mode	Separation	Pre-validation	Data locality
Silent computational errors	✓	✓	
Privacy violations			✓
Reproducibility failures		✓	
Unauditable reasoning	✓		

Note: Check marks indicate which of the three principles (Section 2.2) addresses each failure mode identified in Section 2.

2.4. Relationship to existing ideas

Chat-first analytics applies established techniques to a new domain. Tool-augmented language models (Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda, and Scialom 2023; Yao, Zhao, Yu, Du, Shafran, Narasimhan, and Cao 2023) can reliably select and invoke external tools when provided with appropriate schemas (Qu, Dai, Wei, Xu, Cai, Wang, Yin, and Wen 2025; Liu, Huang, Zeng, Hao, Yu, Li, Wang, Gan, Liu, Yu, Wang, Wang, Ning, Hou, Wang, Wu, Wang, Liu, Wang, Tang, Tu, Shang, Jiang, Tang, Lian, Liu, and Chen 2025). The Model Context Protocol (MCP) (Anthropic 2024) standardizes how language models discover and invoke tools, providing the communication layer between interpretation and computation.¹

The interface innovation has precedents. The Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973) simplified model specification in the 1970s. Text-to-SQL systems (Yu *et al.* 2018) demonstrated natural language as a viable interface to structured computation. AutoML systems (Feurer, Klein, Eggenberger, Springenberg, Blum, and Hutter 2015) automated model selection, though without conversational interaction. Chat-first analytics trades

¹MCP focuses on model-to-tool integration. Complementary protocols address agent-to-agent communication: Google’s Agent2Agent (A2A) protocol (Google 2025) and IBM’s Agent Communication Protocol (ACP) (IBM Research 2025), which merged under Linux Foundation governance in 2025. For single-agent tool invocation—the pattern used here—MCP is the dominant standard, with adoption by OpenAI, Microsoft, and Google alongside Anthropic.

the precision of formal interfaces for accessibility, accepting that some ambiguity in natural language is the cost of making methods reachable to non-programmers.

Several systems have explored LLM-assisted data analysis, but with a code-generation architecture. Hong, Lin, Liu, Liu, Wu, Li, Chen, Zhang, Wang, Zhang, Zhang, Yang, Zhuge, Guo, Zhou, Tao, Wang, Tang, Lu, Zheng, Liang, Fei, Cheng, Xu, and Wu (2025) present a “Data Interpreter” that generates and executes Python code. Chen, Shen, and Ma (2025) develop an “Econometrics AI Agent” that augments code generation with domain knowledge. Sun, Han, Jiang, Qi, Sun, Yuan, and Huang (2025) describe an agent that orchestrates generated code across analysis stages. All three generate code; none pre-validate implementations or enforce data locality. Table 2 summarizes these differences.

Table 2: Comparison with code-generation approaches

	p2a	Econ. AI	Data Interpr.	LAMBDA
LLM role	Tool selection	Enhanced code gen	Code gen + exec	Agent orchestration
Computation	Pre-validated	LLM-generated	LLM-generated	LLM-generated
Local execution	Yes (Ollama)	No	No	No
Validated	Yes (vs. R)	No	No	No

Note: Econ. AI = Econometrics AI Agent (Chen *et al.* 2025); Data Interpr. = Data Interpreter (Hong *et al.* 2025); LAMBDA (Sun *et al.* 2025). All three generate code; none pre-validate implementations or enforce data locality.

2.5. What chat-first does not solve

Two limitations are fundamental, not fixable by better engineering.

Statistical expertise is still required. Chat-first analytics assists with method selection and execution, not with research design or causal identification. A user who requests “estimate the effect of X on Y” receives a regression, but the system does not validate whether a causal interpretation is warranted. LLM explanations may echo causal language from the user’s prompt even when the analysis provides only associational evidence. Users remain responsible for ensuring that their identification strategy supports their claims.

Method selection is imperfect. Tool selection works well on curated prompts but degrades with naturalistic language, and parameter extraction remains the weakest link. Section 5 provides detailed accuracy numbers across models and task types; users should consult those results before interpreting the polished examples in Section 4.

These numbers mean that chat-first analytics is a useful tool, not an autonomous analyst. Users should verify tool selection, check extracted parameters, and apply domain knowledge to interpret results—the same discipline required when using any statistical software, with the added need to verify that the system understood the request correctly.

3. The software: **prompt2analytics**

prompt2analytics implements the three principles defined in Section 2. This section describes the software architecture, the implemented methods, and the communication flow that con-

nects natural language requests to statistical computation.

3.1. Architecture

The software is organized as a Rust workspace with four crates (Figure 1).

p2a-core Core analytics library implementing all statistical methods in pure Rust. This crate has no external econometrics dependencies; all estimators are implemented from first principles using **ndarray** for matrix operations and **faer** for numerically stable linear algebra. Implementations follow standard textbook algorithms: OLS and robust standard errors follow [Wooldridge \(2010\)](#) and [Cameron and Trivedi \(2005\)](#); panel data methods follow [Baltagi \(2013\)](#); time series methods follow [Hamilton \(1994\)](#); maximum likelihood for discrete choice uses Newton-Raphson as described in [Greene \(2018\)](#). An optional **cuda** feature flag enables GPU acceleration for compute-intensive operations on NVIDIA hardware (Section 6).

p2a-cli Command-line interface producing a standalone **p2a** binary with hierarchical subcommands organized by analytical domain.

p2a-mcp Model Context Protocol server exposing tools for LLM integration. Supports stdio, HTTP, and WebSocket transports.

p2a-dioxus Web application in Rust using Dioxus, compiling to WebAssembly for browser deployment.

All user-facing crates depend on **p2a-core**, ensuring identical numerical results regardless of the interface used.

Quick start

The simplest way to use **prompt2analytics** is through the CLI:

```
# Load data and run OLS with robust standard errors
p2a data load wages.csv --name wages
p2a reg ols wages -y salary -x education experience --robust hc1
```

For LLM-mediated access, configure the MCP server and type in natural language:

```
User: Load wages.csv and regress salary on education and experience
      with robust standard errors.
```

Both pathways invoke the same Rust implementation and produce identical numerical results.

3.2. How separation works in practice

The communication flow (Figure 2) shows how Principle 1 (separation) operates concretely:

1. The user submits a natural language query.

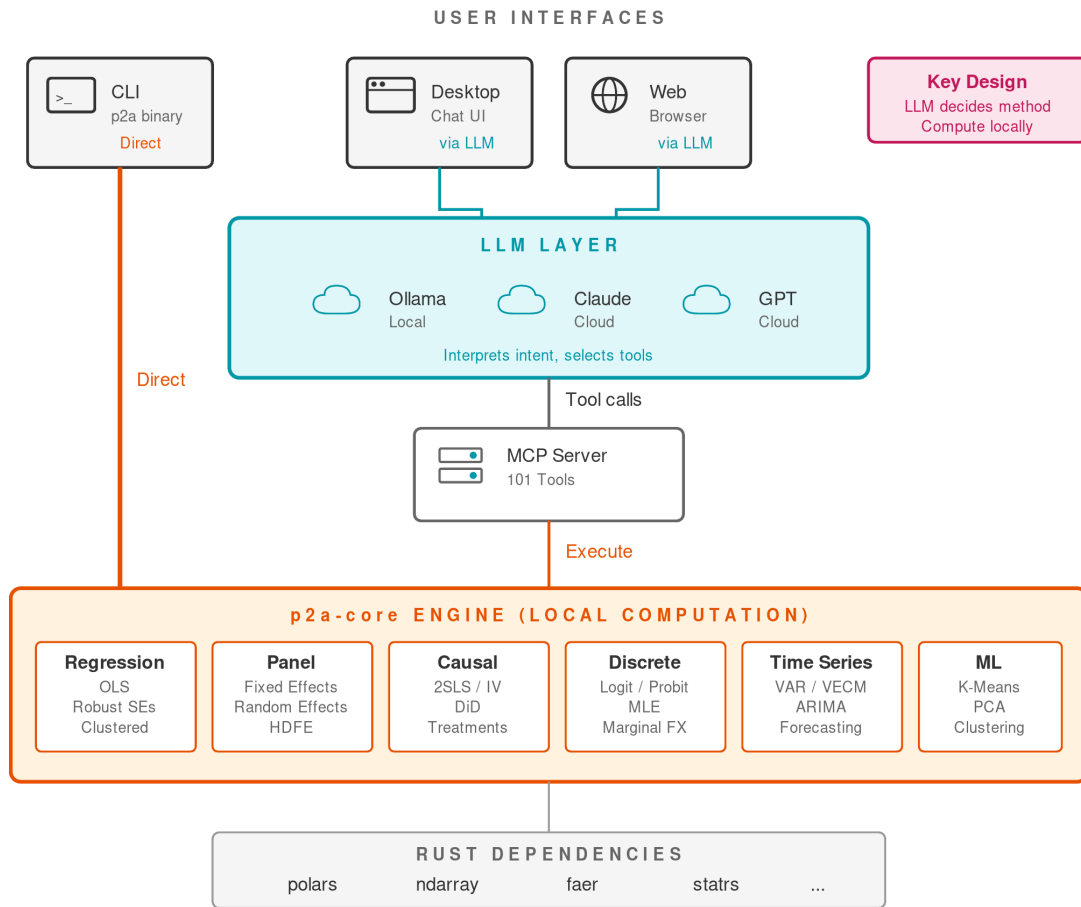


Figure 1: Software architecture of **prompt2analytics**. The CLI accesses the analytics engine directly (orange); chat interfaces route through the LLM layer and MCP server (cyan). All computation occurs locally in **p2a-core**.

2. The LLM interprets the request and generates a structured tool call with parameters—e.g., `regression_ols` with `y_var="salary"`, `robust="hc1"`, and the relevant column names.
3. The MCP server routes the call to the corresponding Rust method in **p2a-core**.
4. The core library performs computation locally.
5. Results (coefficients, standard errors, test statistics) are serialized to JSON and returned through the MCP server.
6. The LLM receives the results and formulates a natural language explanation.

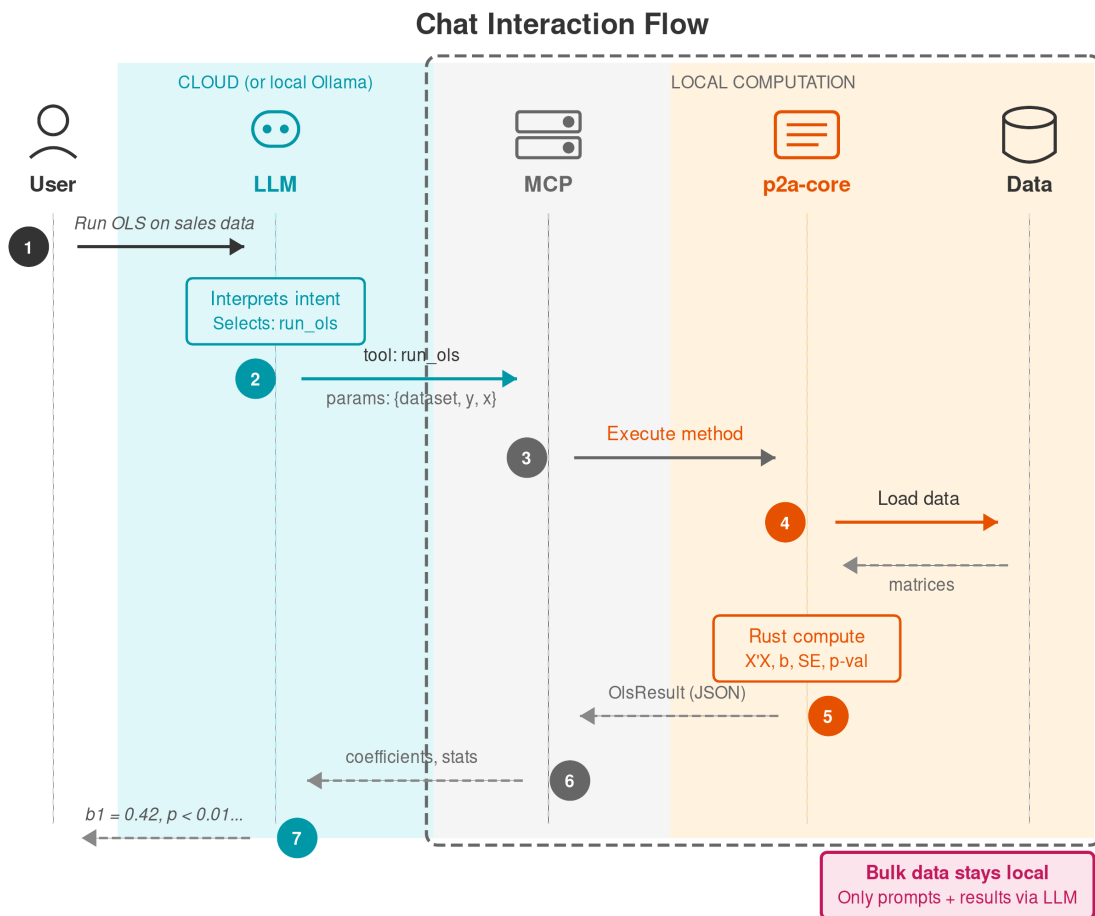


Figure 2: Interaction flow from user request to result. The LLM generates a structured tool call; the MCP server executes the Rust method; results flow back for explanation. Data processing (shaded) stays local.

The LLM never sees raw data. It receives metadata at dataset load time (column names, types, row counts) and aggregated results after computation. Data preview tools (`head`, `sample`) do transmit sample rows; for complete isolation, local LLM deployment via Ollama eliminates all external transmission.

3.3. How pre-validation works

Each tool invokes a fixed Rust function validated against reference R implementations (Section 5). A `LinearEstimator` trait provides a consistent interface across estimators:

```
pub trait LinearEstimator {
    fn coefficients(&self) -> &Array1<f64>;
    fn std_errors(&self) -> &Array1<f64>;
    fn t_values(&self) -> Array1<f64>;    // default impl
    fn p_values(&self) -> Array1<f64>;    // default impl
    fn residuals(&self) -> Array1<f64>;
    fn n_obs(&self) -> usize;
    fn df(&self) -> usize;
}
```

Implementors provide core results; derived statistics (t -values, p -values, confidence intervals) are computed by default implementations, enforced at compile time. Unlike R’s formula interface, the API uses explicit column names, because the LLM has already translated intent into specification.

The `EconError` type provides structured error information (singular matrix, convergence failure, insufficient clusters) that enables the LLM to offer actionable recovery guidance.

Validation uses automated test suites that compare Rust output against reference R packages on shared datasets. Coefficients, standard errors, and test statistics are verified to tolerances between 10^{-4} and 10^{-8} depending on the method. When intentional differences exist—for example, defaulting to HC1 rather than HC3 for robust standard errors, matching Stata’s convention—these are documented and tested separately. The validation approach is described further in Section 5; Appendix J lists method-specific caveats and known differences.

3.4. Implemented methods

The **p2a-core** library implements over 200 methods. Table 3 summarizes coverage; Appendix J provides full documentation.

3.5. The MCP tool layer

The MCP server exposes tools organized into categories that mirror the core library (Table 4). Each tool has a JSON Schema defining required and optional parameters. Schema design minimizes required parameters to reduce LLM errors and provides sensible defaults (e.g., HC1 for robust standard errors). Tool descriptions are written to be interpretable by LLMs, specifying when each tool is appropriate. For example:

```
{
  "name": "regression_ols",
  "description": "Run OLS regression with optional robust SEs",
  "inputSchema": {
    "required": ["dataset", "y_var", "x_vars"],
    "properties": {
```

Table 3: Implemented method categories

Category	Methods
Regression	OLS with HC0–HC3, clustered SEs, HAC, bootstrap, GLS, NLS, LOESS, splines, stepwise, quantile regression (Koenker and Jr. 1978)
Panel data	FE, RE, Hausman, HDFE, panel GLS, Arellano-Bond GMM (Arellano and Bond 1991), panel unit root tests (LLC, IPS, Fisher)
IV	2SLS, first-stage diagnostics, Sargan test, GMM (Hansen 1982)
Causal (design)	DiD, staggered DiD (Callaway and Sant’Anna 2021), Bacon decomposition (Goodman-Bacon 2021), ETWFE (Wooldridge 2021), sharp/fuzzy RD with CCT (Calonico, Cattaneo, and Titiunik 2014), multi-cutoff RD, synthetic control (Abadie, Diamond, and Hainmueller 2010), gsynth (Xu 2017), SCPI (Cattaneo, Feng, and Titiunik 2021)
Causal (treatment)	IPW, AIPW, matching (Ho, Imai, King, and Stuart 2011), CBPS (Imai and Ratkovic 2014), entropy balancing, GBM weighting, TMLE (van der Laan and Rose 2011), CTMLE, LTMLE, DML (Chernozhukov, Chetverikov, Demirer, Duflo, Hansen, Newey, and Robins 2018)
Discrete choice	Logit, probit, multinomial, conditional, mixed, ordered, negative binomial, ZIP/ZINB, hurdle
Time series	ARIMA, VAR, VARMA, VECM, IRF, Granger causality, STL/MSTL, Holt-Winters, structural TS, Kalman filter, GARCH, changepoint
Spatial	SAR, SEM, SAC (Anselin 1988), spatial probit, spatial panel, spatial GMM
Survival	Kaplan-Meier, log-rank, Cox PH, AFT, competing risks
Mediation	Causal mediation, natural effects, sensitivity (Cinelli and Hazlett 2020), E-values, Balke-Pearl bounds
Statistical tests	50+ tests: t -tests, ANOVA, MANOVA, χ^2 , Fisher, Wilcoxon, Kruskal-Wallis, Friedman, Shapiro-Wilk, KS, Tukey HSD

Note: Appendix J provides full documentation. Table A.1 lists unimplemented categories with recommended alternatives.

Table 4: MCP tool categories

Category	Examples	Tools
Data management	<code>load_dataset</code> , <code>describe_dataset</code>	7
Data transformation	<code>munge_filter</code> , <code>munge_join</code> , <code>munge_pivot</code>	19
Regression	<code>regression_ols</code> , <code>regression_diagnostics</code>	10
Panel data	<code>panel_fixed_effects</code> , <code>hausman_test</code>	5
Causal inference	<code>iv_2sls</code> , <code>diff_in_diff</code>	8
Discrete choice	<code>logit</code> , <code>probit</code>	2
Time series	<code>ts_arima_fit</code> , <code>ts_var</code>	6
Statistics	various hypothesis tests	30+
Visualization	<code>viz_scatter</code> , <code>viz_histogram_interactive</code>	7

Note: Tool count per category. Each tool corresponds to one or more **p2a-core** methods and is described by a JSON Schema that LLMs use for parameter extraction.

```

    "dataset": {"type": "string"},
    "y_var": {"type": "string"},
    "x_vars": {"type": "array", "items": {"type": "string"}},
    "robust": {"type": "string", "enum": ["hc0", "hc1", "hc2", "hc3"]},
    "cluster_var": {"type": "string"}
  }
}
}

```

Note the distinction between MCP tools and core methods: tools serve as entry points that may invoke multiple methods. The `regression_ols` tool provides access to OLS with several covariance estimators (HC0–HC3, clustered, HAC), each a distinct method in **p2a-core**. This many-to-one mapping keeps the LLM’s selection task manageable while preserving access to the full library.

3.6. Identification diagnostics

Beyond executing methods, the system generates data-driven warnings when results suggest potential violations of identifying assumptions. Warnings are non-blocking—results are always returned—but highlight conditions that warrant careful interpretation.

For instrumental variables, first-stage F -statistics are checked against the Stock-Yogo threshold of 10 (Stock and Yogo 2005). For staggered difference-in-differences, pre-trend tests are computed automatically. For matching and IPW, balance diagnostics flag standardized mean differences exceeding 0.1 (Austin 2011). For regression discontinuity, effective sample sizes within the bandwidth are checked against minimum thresholds.

Each warning includes a severity level, a diagnostic message, the assumption at risk, and suggested remediation steps. This structured information enables the LLM to provide recovery guidance rather than generic failure messages.

3.7. Why Rust

The choice of Rust follows from the principles. Principle 2 (pre-validation) requires an implementation language where validated methods remain correct across deployments—Rust’s compile-time memory safety guarantees eliminate buffer overflows, use-after-free, and data races that can cause incorrect results in numerical software. Principle 3 (data locality) benefits from compiled performance: the same analytics engine runs efficiently on the user’s machine, not just on cloud servers. A single codebase compiles to standalone binaries, WebAssembly for browsers, and shared libraries for language integration.

The ecosystem provides the necessary components: **polars** for data manipulation (Pola-rs Contributors 2024), **ndarray** for array operations, and **faer** for linear algebra. All computations use 64-bit floating-point arithmetic throughout, matching R’s precision. To date, no econometrics or data analytics library has existed for Rust; **p2a-core** addresses this gap.

3.8. Technical decisions

Several design choices merit brief explanation.

Conversation persistence. The web application uses SurrealDB, an embedded document database, for storing conversation history. Its embedded operation requires no external service; its flexible schema accommodates evolving conversation structures without migrations. Analytical datasets are held in memory during sessions and not persisted, limiting storage to lightweight metadata.

Handling unimplemented methods. When users request methods outside the implemented set, the LLM infers scope from the tool schema. In some cases it maps to the closest available alternative and explains the substitution; in others it reports unavailability. Section 5 evaluates out-of-scope detection rates.

Security. Tools have fixed descriptions embedded in the server binary—external modification requires rebuilding. No arbitrary code execution exists. Tool calls are logged and can be inspected. Dataset access requires explicit loading; the server has no ambient filesystem access. Nevertheless, users should avoid loading untrusted data files that could contain injection payloads, and prefer local LLM deployment for high-security environments.

4. Examples

This section demonstrates **prompt2analytics** through two interfaces: the natural language chat interface powered by large language models, and the command-line interface for scripted workflows. Both interfaces access the same underlying Rust analytics engine.

We use three datasets throughout this section:

- The Grunfeld investment data (Baltagi 2013), a classic panel dataset with 200 observations (10 firms over 20 years) containing firm investment, market value, and capital stock. Included in the **plm** package; we provide it as **grunfeld.csv** in the supplementary materials.
- Quarterly U.S. macroeconomic data (1954–2023) with GDP growth, unemployment, inflation, and the federal funds rate, used for time series examples.
- A simulated cross-sectional dataset demonstrating basic regression analysis with heteroskedasticity.

4.1. Chat interface

The chat interface enables natural language interaction with the analytics engine. Users describe their analysis goals in plain language, and the LLM translates these requests into structured tool calls executed by the MCP server. This interface is useful for exploratory data analysis and for users less familiar with programming syntax. The examples in this section were generated using Claude 3.5 Sonnet via the MCP integration; other models (GPT-4o-mini, Llama 3.3 70B, etc.) produce similar results with minor variations in explanation style.

Loading and exploring data

A typical session begins by loading data and examining its structure:

User: Load the Grunfeld investment dataset from grunfeld.csv and show me what's in it.

I'll load the dataset and describe its contents.

****Dataset loaded: grunfeld****

- Rows: 200

- Columns: 5

Column	Type	Description
firm	Integer	Firm identifier (1-10)
year	Integer	Year (1935-1954)
inv	Float64	Gross investment
value	Float64	Market value of firm
capital	Float64	Stock of plant and equipment

This is a balanced panel dataset with 10 firms observed over 20 years, commonly used to demonstrate panel data methods.

OLS regression with robust standard errors

Cross-sectional regression with heteroskedasticity-consistent standard errors.²

*User: Run a regression of investment on market value and capital stock.
Use robust standard errors.*

Running OLS regression with HC1 robust standard errors...

****OLS Regression Results****

Dependent variable: inv

Variable	Coef	Std Err	t	P> t
(Intercept)	-42.714	11.575	-3.690	0.000
value	0.116	0.007	16.967	0.000
capital	0.231	0.049	4.721	0.000

R-squared: 0.812

Adj. R-squared: 0.810

Observations: 200

Standard errors: HC1 (heteroskedasticity-robust)

Both market value and capital stock are highly significant predictors of investment. A \$1 million increase in market value is associated with approximately \$116,000 more investment, holding capital constant.

²HC1 is the default robust standard error estimator when robust SEs are requested. HC1 applies a degrees-of-freedom correction ($n/(n - k)$) to HC0, improving small-sample performance.

Panel data: Fixed effects estimation

For panel data, users can request fixed effects estimation in natural language:

User: The data has a panel structure. Run a fixed effects regression controlling for firm-specific effects.

Running within-transformation fixed effects estimation...

****Fixed Effects (Within) Estimation****

Dependent variable: inv

Entity fixed effects: firm (10 levels)

Variable	Coef	Std Err	t	P> t
-----	-----:	-----:	-----:	-----:
value	0.110	0.012	9.288	0.000
capital	0.310	0.017	17.867	0.000

R-squared (within): 0.767

Observations: 200

Groups: 10

F-statistic: 309.01 (p-value: 0.000)

After controlling for time-invariant firm characteristics, the coefficient on capital increases from 0.23 to 0.31, suggesting that pooled OLS underestimated the association between capital stock and investment. (Note: This change in coefficient reflects removal of omitted variable bias, not necessarily identification of a causal effect.)

Model comparison: Hausman test

Users can request specification tests without knowing the technical details:

User: Should I use fixed effects or random effects for this analysis?

I'll run a Hausman test to help determine the appropriate specification.

****Hausman Test Results****

H0: Random Effects is consistent and efficient

H1: Fixed Effects is consistent; Random Effects is inconsistent

Chi-squared statistic: 0.0003

Degrees of freedom: 2

P-value: 0.9999

****Recommendation**:** Random Effects

The test fails to reject the null hypothesis. The near-zero chi-squared statistic indicates virtually no systematic difference between the FE and RE coefficient estimates for this dataset.

****Note**:** Extreme p-values like 0.9999 warrant interpretation with caution. Such values can arise from small test statistics (as here), but also from numerical issues or model misspecification. The Hausman test~\citep{Hausman:1978} assumes both estimators are consistent under the null, which requires correct functional form, homoskedastic errors, and no omitted variables correlated with the regressors. The test statistic may be unreliable with small samples, heteroskedasticity, or serial correlation. Users should treat very high p-values as suggestive rather than definitive.

High-dimensional fixed effects

For datasets with multiple categorical dimensions, users can request multi-way fixed effects:

User: Control for both firm and year fixed effects in the regression.

Running two-way fixed effects using alternating projections...

****High-Dimensional Fixed Effects Estimation****

Dependent variable: inv

Fixed effects: firm (10 levels), year (20 levels)

Variable	Coef	Std Err	t	P> t
-----	-----	-----	-----	-----
value	0.118	0.019	6.137	0.000
capital	0.358	0.054	6.579	0.000

R-squared (within): 0.720

Observations: 200

F-statistic: 217.44 (p-value: 0.000)

Regression diagnostics

Users can request diagnostic tests using natural language:

User: Check the regression assumptions for the OLS model.

Running regression diagnostics...

****Jarque-Bera Test (Normality of residuals)****

Statistic: 89.57

P-value: 0.000

Conclusion: Residuals deviate significantly from normality.

```

**Breusch-Pagan Test (Heteroskedasticity)**
Statistic: 55.25
P-value: 0.000
Conclusion: Evidence of heteroskedasticity; robust SEs recommended.

**Durbin-Watson Statistic (Autocorrelation)**
Statistic: 0.358
Interpretation: Positive autocorrelation detected (expected in panel data).

**Variance Inflation Factors**
value: 1.31
capital: 1.31
No multicollinearity detected (VIF < 5).

```

4.2. Command-line interface

The command-line interface provides programmatic access for scripted workflows. Commands follow a hierarchical structure with sessions maintaining state:

```

# Load data and run OLS with robust standard errors
p2a --session analysis.json data load grunfeld.csv --name grunfeld
p2a --session analysis.json reg ols grunfeld \
    -y inv -x value capital --robust hc1

# Panel fixed effects
p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm

# Export session to reproducible script
p2a script export analysis.json -o reproduce.sh

```

Note: Backslash line continuation (\) works in Bash, Zsh, and most POSIX-compatible shells. Windows Command Prompt users should write commands on a single line or use PowerShell's backtick (`) for continuation.

JSON output for programmatic use

All commands support JSON output (`-F json`) for integration with other tools and scripts:

```

p2a -F json reg ols grunfeld -y inv -x value capital --robust hc1

{
  "coefficients": {
    "(Intercept)": -42.714, "value": 0.116, "capital": 0.231
  },

```

```

"std_errors": {
  "(Intercept)": 11.575, "value": 0.007, "capital": 0.049
},
"t_values": {
  "(Intercept)": -3.690, "value": 16.967, "capital": 4.721
},
"p_values": {
  "(Intercept)": 0.000, "value": 0.000, "capital": 0.000
},
"r_squared": 0.812, "adj_r_squared": 0.810,
"n_obs": 200, "cov_type": "HC1"
}

```

This enables integration with analysis pipelines, for example piping results to `jq` for extraction or to Python/R scripts for further processing. The session mechanism records all commands and can export them as an executable shell script for exact reproduction.

Appendix K provides additional CLI examples including panel estimation, hypothesis tests, and two-way fixed effects.

4.3. Comparison with R

For reference, the equivalent R code for the fixed effects estimation is:

```

library(plm)
data("Grunfeld", package = "plm")

# Fixed effects model
fe_model <- plm(inv ~ value + capital,
               data = Grunfeld,
               index = c("firm", "year"),
               model = "within")
summary(fe_model)

# Two-way fixed effects
fe_twoway <- plm(inv ~ value + capital,
                data = Grunfeld,
                index = c("firm", "year"),
                model = "within",
                effect = "twoways")
summary(fe_twoway)

```

The **prompt2analytics** implementation produces numerically equivalent results (see Section 5), while offering the conversational interface that enables users unfamiliar with R syntax to perform the same analyses through natural language.

4.4. Time series analysis: An applied example

To illustrate capabilities beyond panel data, this section demonstrates a time series workflow using quarterly U.S. GDP growth data.

User: Load gdp_quarterly.csv and check for stationarity.

```
**Dataset loaded: gdp**
- Rows: 280 (quarterly, 1954 Q1 to 2023 Q4)
- Columns: date, gdp_growth, unemployment, inflation, fed_rate

**Augmented Dickey-Fuller Test** (gdp_growth)
Statistic: -4.82
P-value: 0.000
```

GDP growth is stationary (rejects unit root). No differencing needed.

User: Fit an ARIMA model to GDP growth. Let the system pick the order.

```
**ARIMA Model Selection** (AIC-based)
Best model: ARIMA(2,0,1)
```

Parameter	Estimate	Std Err
AR(1)	0.312	0.067
AR(2)	0.154	0.063
MA(1)	-0.089	0.072
Intercept	0.782	0.117

```
AIC: 1042.3
Ljung-Box (lag 10): p = 0.482 (no residual autocorrelation)
```

User: Now estimate a VAR with GDP growth, unemployment, and inflation.
I want to see how shocks propagate between these variables.

```
**VAR(2) Estimation** (lag selected by AIC)
Variables: gdp_growth, unemployment, inflation
Observations: 278 (after lag adjustment)
```

```
Granger Causality Tests:
- gdp_growth -> unemployment: F = 8.42, p = 0.000
- unemployment -> gdp_growth: F = 3.17, p = 0.044
- inflation -> gdp_growth: F = 1.89, p = 0.153
```

GDP growth Granger-causes unemployment (and vice versa).
 No significant evidence that inflation Granger-causes GDP growth.

```
Impulse response functions saved to irf_plot.html
(interactive plot with 95% bootstrap confidence bands).
```

This example demonstrates method progression—stationarity testing, univariate modeling, then multivariate analysis—within a single conversation. The LLM maintains context: the third request builds on the loaded dataset and prior results without restating variable names.

4.5. Multi-step workflows

Real-world analysis involves multiple steps: data inspection, handling issues, and specification refinement. The chat interface maintains context across turns, enabling progressive model refinement guided by user questions. The LLM provides interpretive commentary while all computations execute through validated Rust implementations.

Appendix K provides a detailed multi-step workflow example demonstrating data loading, missingness analysis, initial specification, and refinement with fixed effects—illustrating context preservation and proactive issue identification.

4.6. Realistic use

The preceding examples show successful interactions. Real use involves more iteration. This section shows what failures look like, how to recover, and what accuracy to expect (Section 5 provides detailed numbers).

Ambiguous specification

Vague requests prompt the system to ask for clarification rather than guess:

User: Run a regression with firm effects.

I can estimate this model, but need clarification:

1. ****Dependent variable****: Which column should be the outcome?
2. ****Independent variables****: Which predictors should be included?
3. ****Fixed vs Random effects****: Should firm effects be treated as fixed (within estimator) or random (GLS)?

Please specify, e.g., "Regress inv on value and capital with firm fixed effects"

A multi-turn workflow with corrections

The following workflow illustrates common failure modes: wrong method selected, parameters extracted incorrectly, user corrections, eventual success.

*User: Analyze whether the training program affected employee performance.
The data has employees observed before and after training, with a control group that didn't receive training.*

I'll run a t-test comparing performance between training and control groups.

****Two-sample t-test****

Training group mean: 3.82
 Control group mean: 3.45
 Difference: 0.37 (95% CI: [0.21, 0.53])
 p-value: 0.000

The training group shows significantly higher performance.

User: That's not right - I need a difference-in-differences analysis, not just a comparison of means. I have before/after data.

You're right, I apologize. Let me run a difference-in-differences estimation to properly account for the before/after structure.

****Difference-in-Differences Estimation****

Treatment indicator: training_group

Time indicator: post_training

Variable	Coef	Std Err	t	P> t
post_training	0.12	0.08	1.50	0.134
training_group	0.31	0.11	2.82	0.005
post × training (ATT)	0.24	0.09	2.67	0.008

The ATT estimate is 0.24 performance points.

User: Good, but I asked for clustered standard errors at the employee level to account for repeated observations.

Let me re-run with clustered standard errors:

****Difference-in-Differences** (Clustered by employee, 847 clusters)**

Variable	Coef	Cluster SE	t	P> t
post_training	0.12	0.11	1.09	0.276
training_group	0.31	0.14	2.21	0.027
post × training (ATT)	0.24	0.12	2.00	0.046

With clustered SEs, the ATT remains significant at the 5% level, though standard errors are larger than the unclustered estimates.

User: Now check for pre-trends to validate the parallel trends assumption.

****Pre-trend Test****

Testing whether treatment and control groups had parallel trends in the pre-treatment period...

Pre-period	Diff	Std Err	t	P> t
-----	-----	-----	-----	-----
t-3	-0.02	0.09	-0.22	0.826
t-2	0.05	0.08	0.63	0.531
t-1	0.01	0.08	0.13	0.900

Joint test (H0: all pre-period diffs = 0):

F-statistic: 0.18, p-value: 0.912

No evidence of differential pre-trends; parallel trends assumption is supported (though not proven) by this test.

This workflow required three correction turns: (1) correcting method selection from a t-test to DiD; (2) adding clustered standard errors; and (3) requesting pre-trend diagnostics. The final result is correct.

What to expect

From our evaluation (Section 5):

- Multi-turn workflows complete 15–90% of the time when tool calls are in conversation history
- Naming variables directly (“regress inv on value”) beats pronouns (“use the same variables”)
- Check extracted parameters before trusting output—parameter extraction F1 is only 41–59%

The remedy for most failures is to keep tool calls in conversation history, and verify what the model extracted.

4.7. Error handling and recovery

The system handles common failure modes with explanatory guidance. Two examples illustrate the approach.

Multicollinearity detection

User: Add years_since_hire to the model alongside tenure.

****Error: Perfect multicollinearity detected****

The design matrix is rank-deficient. Column `years\_since\_hire` is a linear combination of existing columns (correlation with tenure: 1.000).

****Suggested actions:****

1. Remove one of the collinear variables
2. If both are theoretically important, consider which better captures your construct of interest

Convergence failure

User: Fit a logit model with department fixed effects.

```
**Warning: Convergence issues detected**

**Issue:** Perfect prediction in some groups
- Department "Legal" has 0% bonus rate (23 observations)

**Options:**
1. Drop singleton departments
2. Use a linear probability model (robust to separation)
3. Use penalized likelihood (Firth correction)
```

Running linear probability model with department FE instead...

The system detects problems before estimation fails, explains statistical issues rather than computational symptoms, and provides actionable suggestions. When requests exceed capabilities, it clearly states limitations and points to external tools. Appendix K provides additional error handling examples including small cluster warnings and out-of-scope request handling.

These examples illustrate what the system can do; the next section measures how reliably it does so. We evaluate tool selection accuracy across 12 models, parameter extraction quality, and numerical correctness of the underlying Rust implementations.

5. How well does it work?

Two kinds of accuracy matter: does the LLM select the right tool with the right parameters? And does the Rust implementation produce correct numerical results? This section evaluates both.

5.1. Tool selection evaluation

We evaluated LLM performance on 87 test cases spanning regression, panel data, causal inference, discrete choice, time series, hypothesis testing, machine learning, and visualization. Each case presents a natural language prompt; the model must select the appropriate tool from the **prompt2analytics** schema. Responses are scored as exact (correct tool), acceptable (valid alternative), category (correct domain, wrong method), or failed. Accuracy is computed as $(\text{exact} + \text{acceptable}) / \text{total}$.

Evaluation methodology and reproducibility

All evaluations used temperature = 0 for deterministic outputs. The system prompt instructs the model to select a single tool from the provided MCP schema and return a structured JSON response with tool name and parameters. The complete system prompt is provided in the supplementary materials file `llm_eval_prompts.txt`; the summary form is:

“You are an assistant that selects statistical tools. Given a user request, choose exactly one tool from the available schema and provide the required parameters. Respond with valid JSON containing ‘tool_name’ and ‘parameters.’”

Table 5 lists the model versions used for evaluation. All models were accessed through cloud APIs (Anthropic, OpenAI, or OpenRouter). API-served models may change over time, so exact replication of results may not be possible. For local deployment, the same open-source models can be served via Ollama with fixed quantization, improving reproducibility.

Table 5: Models evaluated

Model	Version ID	Provider
Claude 3.5 Haiku	claude-3-5-haiku-20241022	Anthropic API
GPT-4o-mini	gpt-4o-mini-2024-07-18	OpenAI API
GPT-5 Nano ^a	gpt-5-nano-2025-08-07	OpenAI API
GPT-4.1 Nano	gpt-4.1-nano-2025-04-14	OpenAI API
Qwen 2.5 72B	qwen-2.5-72b-instruct	OpenRouter
Llama 3.3 70B	llama-3.3-70b-instruct	OpenRouter
Mistral Small 24B ^b	mistral-small-24b-instruct-2501	OpenRouter
DeepSeek ^a	deepseek-chat	OpenRouter
Nemotron Nano 9B	nemotron-nano-9b-v2	OpenRouter
Qwen3 8B	qwen3-8b	OpenRouter
Qwen 2.5 7B ^b	qwen-2.5-7b-instruct	OpenRouter
Ministral 3B	ministral-3b	OpenRouter

Note: Accessed January 2026. Open-source models served through OpenRouter.

^a Single-turn evaluation only. ^b Extended evaluation only. API-served models may change over time.

Figure 3 presents overall accuracy by model. The results demonstrate that tool selection—unlike code generation—is achievable even by smaller models. Claude 3.5 Haiku and Qwen 2.5 72B (an open-source model) achieve 100% accuracy. Notably, Ministral 3B, a model small enough to run on consumer hardware with only 6 GB VRAM, achieves 90.8% accuracy, making it viable for local deployment scenarios where hardware constraints preclude larger models.

Figure 4 shows accuracy by category. Performance is uniformly high across most categories, with errors concentrated in edge cases requiring fine distinctions between related methods. For instance, Ministral 3B shows lower accuracy on time series (66.7%) where distinguishing between VAR, VARMA, and VECM specifications proves challenging for the smaller model. Specific confusion patterns include: selecting VAR when VECM is appropriate (failing to recognize cointegration requirements), choosing ARIMA for multivariate series that require VAR, and defaulting to simpler univariate methods when the prompt describes multiple interrelated series.

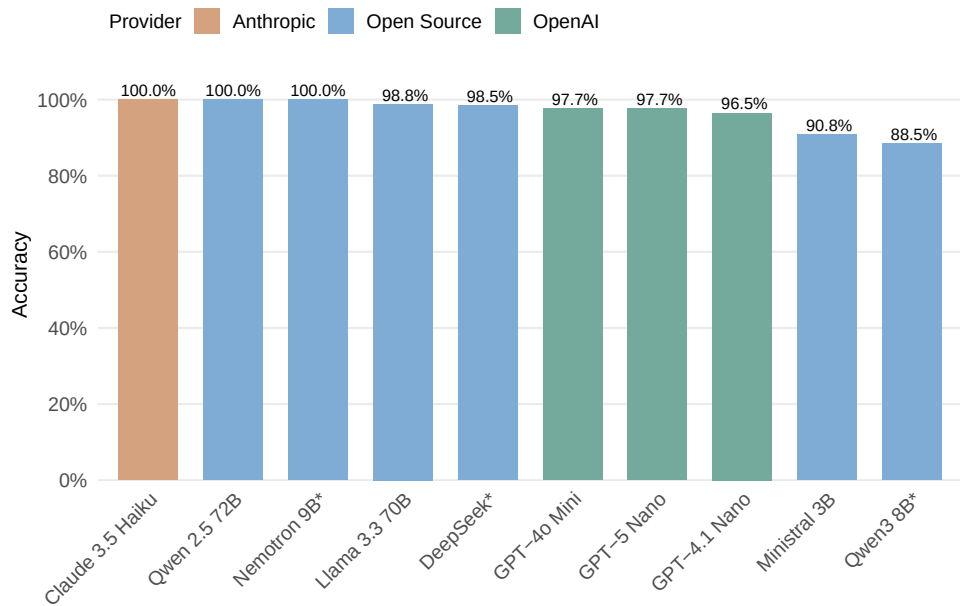


Figure 3: Tool selection accuracy across models (87 tests). Accuracy = (exact + acceptable) / total. Even Minstral 3B (3B parameters) exceeds 90%.

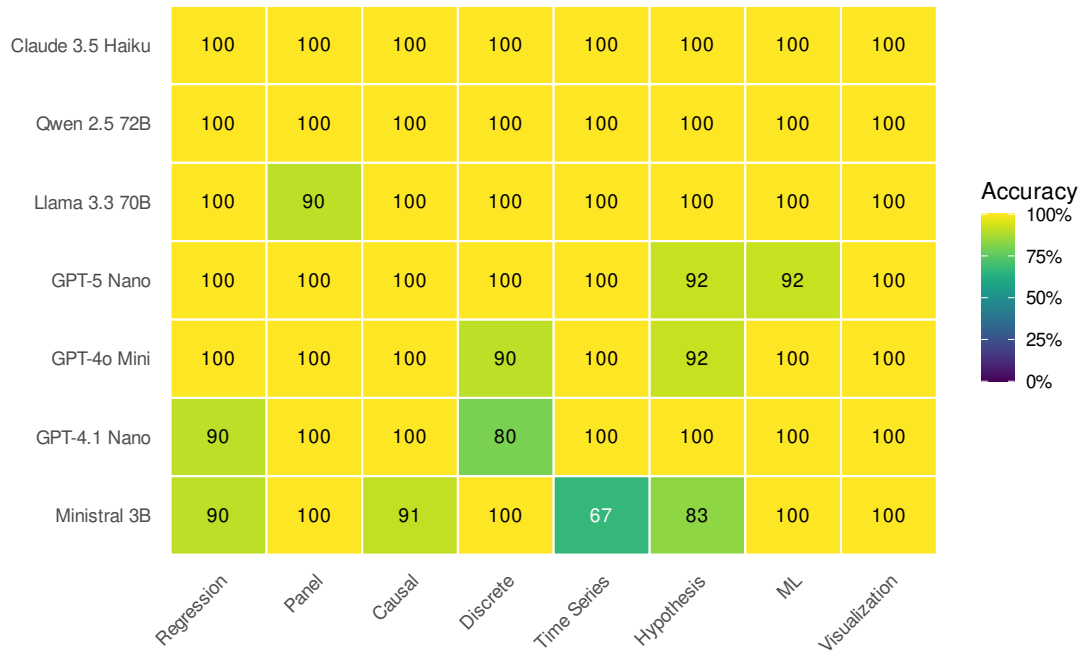


Figure 4: Accuracy by test category. Darker cells indicate lower accuracy. Errors concentrate in ambiguous method selection.

Tables B.1 and B.2 in Appendix B provide detailed breakdowns of model performance and per-category accuracy. Appendix D reports API response latencies, though these figures reflect cloud-hosted inference and do not directly apply to local deployment scenarios.

5.2. Extended robustness evaluation

The single-turn evaluation on curated prompts establishes baseline tool selection accuracy but leaves several questions unanswered: How do models perform across multi-turn conversations? Do they handle naturalistic language with typos and informal phrasing? Can they extract parameters correctly and interpret results accurately? To address these gaps, we conducted an extended evaluation across five dimensions using a partially overlapping set of 10 models (Table C.1), substituting Mistral Small 24B and Qwen 2.5 7B for GPT-5 Nano and DeepSeek to broaden coverage of open-source models.

Multi-turn conversations. We constructed 20 multi-turn conversations totaling 72 turns, where each turn builds on previous context (e.g., “Now use robust standard errors instead”). The results reveal a strong dependence on conversation history: accuracy depends heavily on what goes into the conversation history. When models can see their previous tool calls and results, they correctly resolve references like “those results” or “the previous model.” Without this context, they cannot. GPT-4o-mini with full context achieves 97.2% per-turn accuracy and 90% conversation completion. Qwen3 8B reaches 83.3% per-turn accuracy and 60% conversation completion. Stripping out the tool-call history, however, causes accuracy to drop to 47–60%. The implication is clear: conversation history must include tool invocations, not just the text exchange.

Naturalistic prompts. We tested 99 prompts featuring realistic variations: typos (“regression”), informal language (“run a quick OLS”), domain jargon (“estimate the ATT”), verbose descriptions, and ambiguous phrasing. Accuracy ranges from 58–70%, with Claude 3.5 Haiku achieving the highest robustness (69.7%). Models handle typos well (65–69% accuracy) but struggle more with ambiguous specifications (42–58%).

Parameter extraction. Beyond tool selection, LLMs must extract variable names, options, and modifiers. We evaluated 49 parameter extraction tasks, measuring precision, recall, and F1 score. Average F1 ranges from 41–59% across models, with Qwen 2.5 72B achieving the highest score (59.1%). Common errors include omitting optional parameters, misidentifying column names from context, and confusing similar parameter types.

Interpretation accuracy. We evaluated whether models correctly interpret statistical output across 26 test cases covering coefficient interpretation, significance testing, and diagnostic results. Models achieve 68–87% accuracy, with GPT-4.1 Nano performing best (87.2%). Errors typically involve misinterpreting log-transformed coefficients or overlooking important caveats about statistical significance.

Out-of-scope detection. When users request methods outside the implemented set (e.g., Bayesian inference, structural equation modeling), the system should either decline gracefully or suggest alternatives. Detection rates range from 40–60%, with most undetected cases resulting in selection of a superficially similar but incorrect method.

Table 6 summarizes these results. Appendix C provides per-model breakdowns.

Figure C.1 in Appendix C visualizes the multi-dimensional performance profile for each model.

What this means. Context management matters more than model choice. With tool calls

Table 6: Accuracy summary across evaluation dimensions

Metric	Range	Best model
Single-turn tool selection	90.8–100%	Qwen 2.5 72B, Claude 3.5 Haiku
Multi-turn accuracy	47–97%	GPT-4o-mini (97.2%)
Conversation completion	15–90%	GPT-4o-mini (90%)
Naturalistic prompt robustness	58–70%	Claude 3.5 Haiku (69.7%)
Parameter extraction F1	41–59%	Qwen 2.5 72B (59.1%)
Interpretation accuracy	68–87%	GPT-4.1 Nano (87.2%)
Out-of-scope detection	40–60%	Nemotron Nano 9B (60%)

Note: Ranges span 10 models. Single-turn accuracy on curated benchmarks overstates practical capability. Multi-turn results use enhanced context (tool calls in history). Appendix C provides per-model breakdowns.

in history, even Qwen3 8B (an 8-billion parameter open model) achieves 83% multi-turn accuracy. Without it, 70-billion parameter models fail to exceed 60%. Parameter extraction remains the weak point: F1 scores of 41–59% mean users should check that the model extracted the right variable names and options before trusting results. For practical deployment: include tool calls in conversation history, and verify parameters before interpreting output.

5.3. Failure modes

Understanding system behavior under failure conditions is essential for informed use. Three categories of failure merit discussion.

Method misselection. When the LLM selects an inappropriate statistical method, the system proceeds with the selected method and returns results. Users must recognize the mismatch between request and execution. Common patterns include ambiguous specifications (“regression with firm effects” could invoke OLS, fixed effects, or random effects), similar method confusion (VAR/VECM/VARMA), and capability boundary cases where requests for unimplemented methods map to superficially similar available tools. Users can mitigate misselection by specifying methods explicitly rather than describing goals abstractly.

Parameter extraction errors. Even when the correct method is selected, the LLM may extract incorrect parameters: variable names that do not exist in the dataset, omitted specification options (robust SEs, clustering), or incorrect defaults. The dataset schema—column names and types—is provided to the LLM as context, which substantially reduces variable name errors. Explicit phrasing (“with HC1 robust standard errors”) yields more reliable extraction than implicit expectations.

Categorical variable handling. Categorical (string) variables are automatically converted to dummy variables using treatment coding (one level omitted as reference). The system detects string columns, creates indicator variables, reports the reference category, and warns when categorical variables have many levels (> 50).

5.4. Numerical validation against R

The preceding subsections evaluated the LLM layer. This subsection evaluates the computation layer: does the Rust implementation produce the same numbers as established R packages?

We validate all methods against reference implementations using sample-size-dependent tolerances (looser for small samples where ill-conditioning amplifies differences, tighter for large samples). Reference packages include `lm()`/`sandwich` for OLS, `plm/lfe` for panel data, `glm()` for discrete choice, and `forecast` for time series. All validation tests use fixed random seeds (seed = 42) for reproducibility; reference R scripts are provided in the supplementary materials.

OLS and robust standard errors

The Longley dataset (Longley 1967), with its extreme multicollinearity (condition number $> 10^9$), provides a stringent test. Table 7 shows coefficients match R’s `lm()` to at least 8 significant digits.

Table 7: OLS validation: Longley dataset

Coefficient	R <code>lm()</code>	prompt2analytics	Difference
(Intercept)	−3482.259	−3482.259	$< 10^{-8}$
GNP.deflator	0.01506	0.01506	$< 10^{-8}$
GNP	−0.03582	−0.03582	$< 10^{-8}$
Unemployed	−0.02020	−0.02020	$< 10^{-8}$
Armed.Forces	−0.01033	−0.01033	$< 10^{-8}$
Population	−0.05110	−0.05110	$< 10^{-8}$
Year	1.82915	1.82915	$< 10^{-8}$
R^2	0.9955	0.9955	$< 10^{-4}$

Note: $n = 16$, $k = 6$. The Longley dataset’s extreme multicollinearity (condition number $> 10^9$) provides a stringent numerical test. All coefficients match to at least 8 significant digits.

Heteroskedasticity-consistent standard errors (HC0–HC3) are validated against **sandwich** (Zeileis 2004). HC0 and HC1 match to $< 10^{-8}$; HC2 and HC3 match to $< 10^{-7}$, reflecting algorithmic differences in leverage computation (QR decomposition vs. direct matrix multiplication). Both approaches are numerically valid; the discrepancy is floating-point accumulation, not methodology.

Panel data

Fixed effects coefficients on the Grunfeld dataset (10 firms, 20 years, $n = 200$) match **plm** (Croissant and Millo 2008) to $< 10^{-6}$; the within R^2 matches to four decimal places. The Hausman test statistic matches to $< 10^{-4}$. Two-way HD FE via the MAP algorithm (Gaure 2013) matches **lfe**’s `felm()` to $< 10^{-5}$.

Validation coverage

Beyond the methods shown above, the validation suite covers over 200 methods with more than 1,500 individual test cases (334 specifically targeting validation against R reference

implementations). Statistical tests (t -tests, ANOVA, chi-squared, Fisher’s exact, Wilcoxon, Shapiro-Wilk, Kolmogorov-Smirnov) are validated against R’s **stats** package. Discrete choice models (logit, probit) match **glm()** coefficients and standard errors. Time series methods (VAR, ARIMA) match **vars** and **forecast**. All validation tests are integrated into continuous integration and can be run via `cargo test -p p2a-core`.

Appendix H documents tolerance thresholds by method category and sample size. Appendix J lists known differences from reference implementations (default robust SE type, convergence tolerances, R^2 variant selection)—none affecting the validity of statistical inference.

5.5. Feature comparison with R packages

Performance is one dimension of software quality; feature completeness and ecosystem integration matter equally. Table 8 compares **prompt2analytics** against established R packages across dimensions beyond execution speed.

Table 8: Feature comparison with R packages

Feature	p2a	fixest	lfe	plm
<i>Panel data methods</i>				
Within (FE) estimator	✓	✓	✓	✓
Random effects (GLS)	✓	—	—	✓
High-dimensional FE	✓	✓	✓	—
Multi-way clustering	2-way	n -way	2-way	2-way
Hausman test	✓	—	—	✓
<i>Standard errors</i>				
HC0–HC3 robust	✓	✓	✓	✓
Driscoll-Kraay	✓	✓	—	✓
Newey-West HAC	✓	✓	✓	✓
Conley spatial	—	✓	✓	—
<i>Causal inference</i>				
Difference-in-differences	✓	✓	—	—
Callaway-Sant’Anna	✓	✓	—	—
IV/2SLS	✓	✓	✓	✓
Weak instrument tests	F-stat	✓	✓	—
<i>Integration</i>				
LLM/MCP interface	✓	—	—	—
Local execution (no R)	✓	—	—	—
tidyverse compatible	—	✓	—	✓

Note: ✓ = supported; — = not available. The primary remaining gap is Conley spatial standard errors. **p2a** = **prompt2analytics**.

For core panel methods, **prompt2analytics** provides comparable coverage to specialized packages; the primary remaining gap is Conley spatial standard errors. The distinguishing features are the natural language interface and fully local execution without R dependencies. Users requiring maximum performance for high-dimensional fixed effects should note that

fixest (Bergé 2018) likely outperforms **prompt2analytics** due to its highly optimized C++ implementation. **prompt2analytics** complements rather than replaces specialized tools. Having established the system’s accuracy and feature coverage, the next section examines whether it can run efficiently—and entirely locally—on consumer hardware.

6. Performance and deployment

The three principles require software fast enough to run locally and models small enough to fit on consumer hardware. This section presents performance benchmarks against R and describes fully local deployment configurations where all components—including the language model—run on the user’s machine.

6.1. Performance comparison with R

Table 9 summarizes performance results across representative methods at $n = 10,000$ observations, where computational costs dominate startup overhead. Speedup factors vary by method category and sample size, reflecting differences in algorithmic complexity and the relative weight of interpreter overhead versus numerical computation.

Table 9: Performance comparison: Rust vs. R

Category	Method	n	R	p2a	Speedup
Regression	OLS	100	812	42	19×
	OLS	10,000	2,181	924	2.4×
	OLS + HC1	10,000	25,224	1,571	16×
Panel	Fixed effects	100	4,901	27	184×
	HDFE (2-way)	5,000	26,657	1,159	23×
Discrete	Logit	1,000	2,176	401	5.4×
	Probit	1,000	2,760	2,347	1.2×
Time series	ARIMA(1,1,1)	500	4,783	521	9.2×
	MSTL	500	1,853	219	8.5×
ML	K-means ($k = 3$)	1,000	1,274	587	2.2×
	PCA	1,000	246	67	3.7×

Note: Median execution time in μ s. Speedup = R time / Rust time. Large small-sample speedups (e.g., 184× for panel FE at $n=100$) primarily reflect interpreter overhead rather than computational advantages.

Speedup factors are largest for methods with significant per-call overhead in R. Panel fixed effects at $n = 100$ is 184× faster, but this primarily reflects R’s interpreter overhead (formula parsing, factor handling) rather than fundamental computational advantages. At $n = 10,000$, where matrix operations dominate, speedups converge to 2–6×. Users should focus on the larger-sample results as indicative of typical performance.

Robust standard error computation shows consistent 13–16× speedups across sample sizes. Probit shows only 1.2× because both implementations spend most time evaluating the stan-

dard normal CDF.

At $n = 100,000$ (Table 10), the pattern consolidates: panel fixed effects $6.1\times$, OLS with robust SEs $5.1\text{--}5.5\times$, K-means $4.5\times$, logit $3.1\times$. PCA shows R faster ($0.3\times$) due to highly optimized LAPACK eigenvalue routines. Complete benchmark results are in Appendix H.³

Table 10: Full benchmark results at $n = 100,000$

Category	Method	n	R (ms)	Rust (ms)	Speedup
Discrete	Logit	100,000	230.1	75.0	$3.1\times$
Discrete	Probit	100,000	180.8	306.4	$0.6\times$
ML	Kmeans	100,000	592.4	131.7	$4.5\times$
ML	Pca	100,000	18.1	68.5	$0.3\times$
ML	Dbscan	100,000	8762.3	71707.0	$0.1\times$
Munging	Group By	100,000	30.3	18.6	$1.6\times$
Munging	Sort	100,000	12.8	23.4	$0.6\times$
Munging	Filter	100,000	9.4	24.8	$0.4\times$
Munging	Standardize	100,000	6.1	15.9	$0.4\times$
Munging	Lag	100,000	0.7	12.9	$0.1\times$
Munging	Select	100,000	0.0	12.8	$0.0\times$
Panel	Panel Fe	100,000	194.8	31.7	$6.1\times$
Panel	Panel Re	100,000	451.1	114.0	$4.0\times$
Regression	Ols+Hc2	100,000	159.8	29.3	$5.5\times$
Regression	Ols+Hc0	100,000	137.2	26.0	$5.3\times$
Regression	Ols+Hc3	100,000	139.6	27.2	$5.1\times$
Regression	Ols+Hc1	100,000	142.3	27.9	$5.1\times$
Regression	Ols	100,000	23.1	28.0	$0.8\times$

Note: Median execution time in ms. Speedup = R time / Rust time. Methods with speedup $<1\times$ are faster in R, typically because CLI startup overhead dominates trivial computations.

Sources of performance differences

The performance advantages stem from Rust’s ownership system (stack allocation and in-place mutation without garbage collection), the **faer** library’s SIMD-optimized linear algebra, the column-based API that eliminates formula parsing, and the single-language design that avoids data serialization between components. These advantages diminish as sample size increases: for $n > 100,000$, both implementations spend most time in optimized BLAS routines, and speedup factors converge toward $2\text{--}3\times$.

Optional GPU acceleration

For users with CUDA-capable hardware, **p2a-core** provides optional GPU acceleration via cuBLAS and cuSOLVER. The `cuda` feature flag enables GPU code paths at compile time; at runtime, a singleton context attempts CUDA initialization and falls back silently to CPU if

³A notable omission is **fixest** (Bergé 2018), whose optimized C++ implementation outperforms both **plm** and **prompt2analytics** for HDFE problems at large scale.

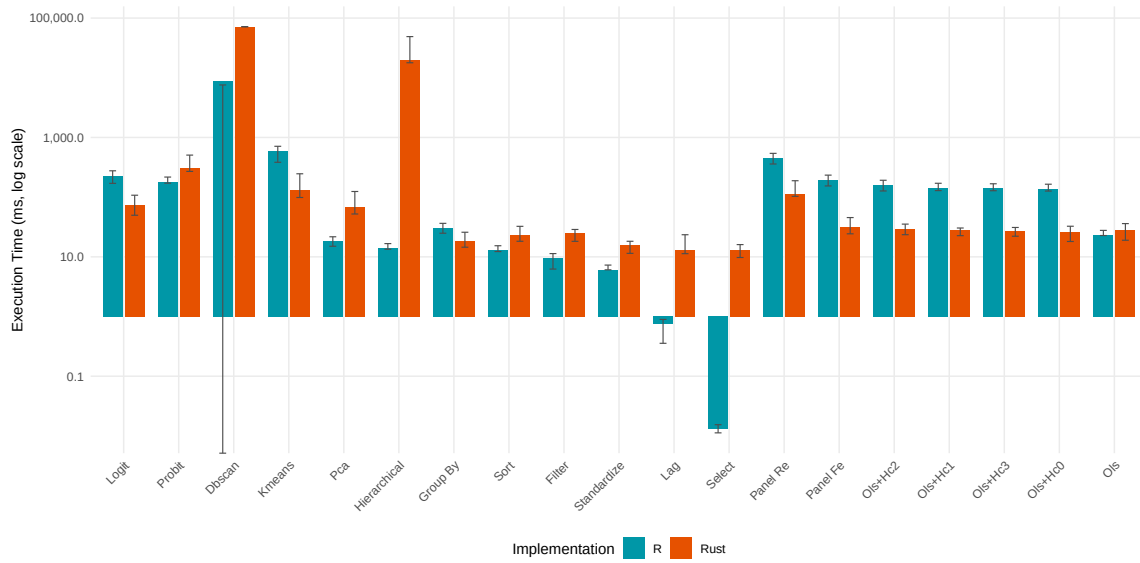


Figure 5: Execution time distributions at $n = 10,000$ (log scale). **prompt2analytics** (orange) shows lower variance than R (blue), reflecting the absence of garbage collection pauses.

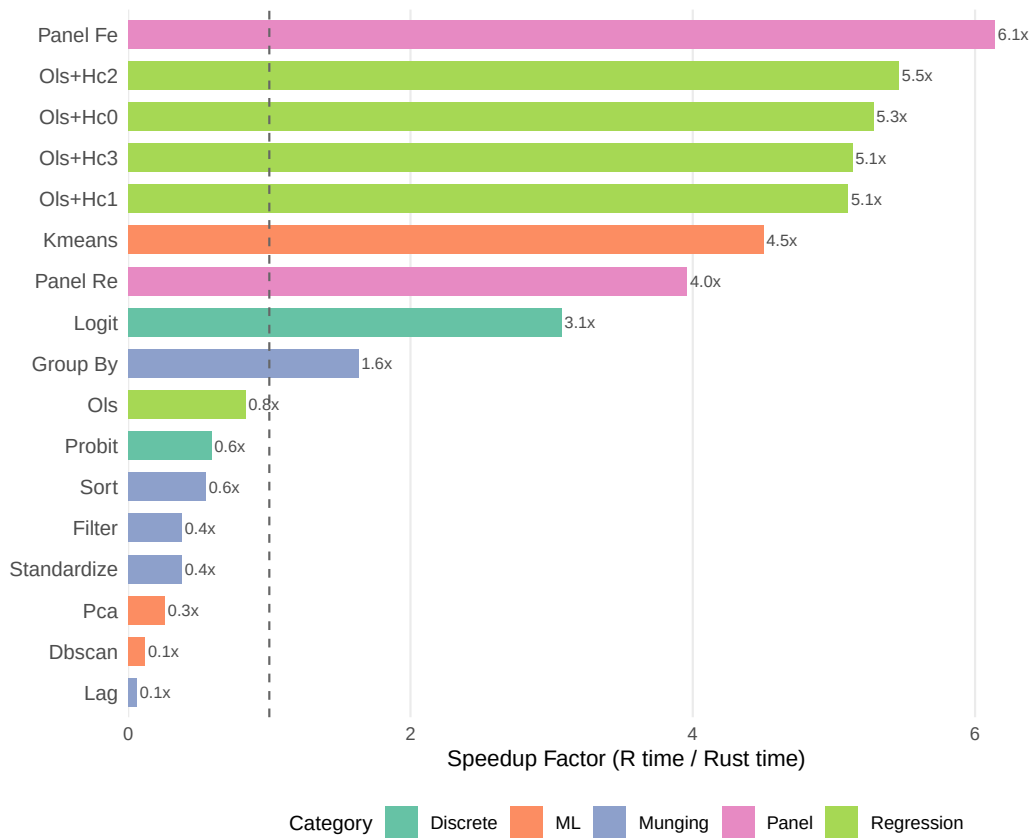


Figure 6: Speedup factors by method at $n = 100,000$. The vertical line at $1\times$ indicates parity.

no device is available. Public function signatures are unchanged—users do not modify their code.

Dispatch mechanism. Not all operations benefit from GPU offloading. Transfer overhead between host and device memory dominates for small problems, and bandwidth-bound operations (e.g., matrix-vector products) can be faster on CPU even for large n . The dispatch logic therefore checks two conditions before routing to GPU: a minimum problem dimension (e.g., $k \geq 30$ columns for $\mathbf{X}'\mathbf{X}$) and a minimum computational intensity (e.g., $nk^2 \geq 2 \times 10^7$). Both thresholds are configurable via environment variables, allowing users to tune dispatch for their specific hardware. A shape guard additionally prevents GPU dispatch for tall-skinny matrix multiplications, where the CPU path is substantially faster.

Accelerated operations. Table 11 summarizes the operations with GPU code paths and observed speedups on an NVIDIA DGX Spark (Grace Blackwell, 12 GB VRAM, unified memory). The largest gains appear in $\mathbf{X}'\mathbf{X}$ computation (up to $7\times$ for $k \geq 100$) and PCA via the covariance path (up to $13\times$ for tall matrices). K-means distance computation and sandwich estimators for heteroskedasticity-robust standard errors see more modest improvements of $2\text{--}3\times$.

Table 11: GPU-accelerated operations and observed speedups

Operation	Use in	Speedup	Condition
$\mathbf{X}'\mathbf{X}$ (DGEMM)	OLS, panel FE/RE, IV, robust SEs	$2\text{--}7\times$	$k \geq 30$
PCA covariance	Dimensionality reduction	$4\text{--}13\times$	$n > 4k$
K-means distances	Clustering	$2\text{--}3\times$	$d \geq 20$
Sandwich $\mathbf{X}'\boldsymbol{\Omega}\mathbf{X}$	HC0–HC3 robust SEs	$2\text{--}3\times$	$k \geq 30$

Note: Measured on NVIDIA DGX Spark (Grace Blackwell, unified memory). Speedups are relative to the CPU path on the same machine (ARM Cortex-X925 with OpenBLAS). Results will differ on discrete GPUs where host–device transfer costs are higher.

Scope and limitations. Most econometric applications involve moderate numbers of regressors ($k < 30$), where CPU execution is faster. GPU acceleration is therefore not part of the default build; it targets high-performance computing scenarios with many covariates, high-dimensional clustering, or large-scale bootstrap procedures. The implementation exploits a layout convention—`ndarray`’s row-major storage appears as a transposed column-major matrix to cuBLAS—that avoids data reordering on the host side. Operations that are bandwidth-bound rather than compute-bound ($\mathbf{X}'\mathbf{y}$ via DGEMV) remain on CPU, where OpenBLAS is consistently faster. The feature is currently limited to NVIDIA GPUs via CUDA; extension to other backends (e.g., Vulkan compute, Metal) is left for future work.

6.2. Fully local deployment

While cloud-hosted language models offer convenience, many use cases require complete data isolation. The fully local configuration runs all components on the user’s machine via Ollama (Ollama Inc. 2024), eliminating all external network communication.

Model selection

The evaluation results (Section 5) inform practical model recommendations. Open-source models can match proprietary alternatives: Qwen 2.5 72B achieves 100% accuracy on our benchmark, identical to Claude 3.5 Haiku. Users requiring complete data isolation need not sacrifice tool selection quality.

Table 12 summarizes recommendations by usage pattern. Table 13 provides hardware requirements.

Table 12: Model recommendations by use case

Use case	Guidance	Recommended model
Single-turn analyses	Even small models perform adequately for one-off regressions or hypothesis tests	Ministral 3B (90.8%)
Multi-turn workflows	Larger models maintain context better; prefer 70B+ for iterative specification refinement	Llama 3.3 70B (59.7%)
Parameter-sensitive	When accurate variable name and option extraction is critical	Qwen 2.5 72B (F1: 59.1%)
Result interpretation	When LLM explanations of output matter; requires cloud API	GPT-4.1 Nano (87.2%)

Note: Accuracy figures are from the evaluations in Section 5. Percentages denote single-turn tool selection accuracy except where noted.

Table 13: Local deployment: models and hardware

Model	Params	Storage	RAM	Acc. (%)	Notes
<i>High accuracy, workstation hardware</i>					
Qwen 2.5 72B	72B	42 GB	48 GB	100.0	Matches proprietary models
Llama 3.3 70B	70B	40 GB	48 GB	98.9	Strong open-source option
<i>Balanced accuracy, consumer hardware</i>					
Llama 3.1 8B	8B	4.7 GB	8 GB	—	BFCL benchmark leader
Qwen 2.5 7B	7B	4.4 GB	8 GB	—	Strong tool-calling accuracy
<i>Viable accuracy, minimal hardware</i>					
Ministral 3B	3B	2.0 GB	6 GB	90.8	Smallest viable model

Note: Accuracy figures are from the 87-test benchmark. Storage refers to Q4_K_M quantization (4-bit). RAM is memory during inference, typically 1.5–2× storage due to KV cache and computational buffers.

Ministral 3B demonstrates that even very small models achieve viable accuracy. With only 3 billion parameters, it runs on virtually any modern laptop—approximately 6 GB VRAM for GPU inference, or CPU-only with 8 GB RAM. The 9.2% error rate concentrates in genuinely ambiguous cases. The 7–8B parameter class offers a practical middle ground, fitting on consumer GPUs with 8+ GB VRAM (e.g., NVIDIA RTX 3070/4070) or Apple M-series devices with 16 GB unified memory.

Hardware requirements

Beyond LLM memory, users must budget RAM for dataset operations. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM; a minimal laptop setup (Minstral 3B, small datasets) works within 16 GB. Appendix E.8 provides detailed RAM requirements by dataset size (Table E.1) and system configurations for common deployment scenarios (Table E.2).

6.3. Privacy guarantees

The fully local configuration provides complete data isolation. No network requests are made to external services. All model inference occurs on the user’s hardware, dataset contents never leave the machine, and query history remains in local storage. This architecture is suitable for analysis of sensitive data where regulatory requirements (GDPR, HIPAA, or institutional review board protocols) or organizational policy prohibit cloud processing. Users should verify that their Ollama installation does not enable telemetry or automatic model updates. Appendix F provides detailed data transmission documentation and security best practices. With the performance, deployment, and privacy picture complete, we turn to an overall assessment of the contribution and its limitations.

7. Discussion

7.1. Summary

This paper has described chat-first data analytics—an approach that separates what language models do well (interpreting intent, selecting methods) from what they do poorly (generating correct numerical code). Three principles define the approach: separate interpretation from computation, pre-validate methods rather than generate code, and keep data local.

prompt2analytics demonstrates this approach through an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol for integration with language models. The software implements over 200 methods across regression, panel data, causal inference, discrete choice, time series, and standard statistical tests—all validated against established R packages.

The evaluation shows that tool selection works well (90.8–100% on curated prompts) and that even small models can do it. However, realistic use is more challenging: naturalistic prompts reduce accuracy to 58–70%, parameter extraction achieves F1 scores of only 41–59%, and out-of-scope detection catches only 40–60% of unsupported requests. The single most important implementation detail is including tool calls in conversation history, which raises multi-turn accuracy from ~50% to ~97%.

7.2. Limitations

No user studies. The practical benefits of chat-first interfaces—accessibility, reduced cognitive load, exploratory dialogue—remain hypothesized. Controlled experiments comparing task completion time, error rates, and user satisfaction between chat-first and traditional interfaces are needed. Such studies were beyond the scope of this work but represent the most

important direction for future research.

Parameter extraction is weak. F1 scores of 41–59% mean that roughly half the time, the model misses an option or extracts the wrong variable name. This is the gap between “the system selected the right method” and “the system ran exactly the analysis I wanted.” Users must verify.

Method coverage gaps. While coverage is broad, important categories remain unimplemented: Bayesian inference, Heckman selection, Conley spatial standard errors, and structural equation modeling. Table A.1 in Appendix A documents gaps and recommends alternatives.

Scalability. Data are loaded entirely into memory; datasets exceeding available RAM require external preprocessing.

Evolving API. The MCP schema uses semantic versioning (currently 0.x); the API may change as the protocol and the software mature.

7.3. Future directions

User studies. Controlled experiments measuring task completion rates, time-to-insight, and error detection rates would provide stronger evidence. Such studies should compare chat-first interfaces against traditional code-based workflows across users with varying levels of statistical expertise.

Better parameter extraction. The weakest link in the pipeline deserves targeted improvement: fine-tuning on domain-specific extraction tasks, structured output constraints, or two-pass verification where the model checks its own parameter extraction before execution.

Formal verification. Property-based testing could strengthen correctness guarantees beyond comparison to reference implementations. QuickCheck-style testing for statistical properties (e.g., residuals orthogonal to regressors) would complement numerical validation.

Language integration. Foreign function interfaces to R and Python would let users call the Rust implementations from familiar environments, combining compiled performance with existing analytical workflows.

7.4. Broader implications

prompt2analytics is not a replacement for R, Stata, or Python. It occupies a different niche: users who know what analysis they want but not which software command produces it. A researcher fluent in R has no need for a conversational interface to OLS; a policy analyst who has never opened R might. The system complements existing tools rather than competing with them—and for users who outgrow the conversational interface, every tool call maps transparently to a function that can be called directly from Rust or, once foreign function interfaces are available, from R and Python.

More broadly, chat-first data analytics exemplifies a pattern applicable beyond econometrics: using language models as interpreters between human intent and specialized software (Yao *et al.* 2023; Schick *et al.* 2023). Any domain where users express goals in natural language and systems execute well-defined operations—laboratory automation, geographic information systems, circuit simulation—could adopt the same architecture, particularly as structured protocols like MCP (Anthropic 2024) standardize the interface between models and tools.

The success of this approach depends on a clear division of labor. Language models interpret

and explain; specialized software computes. Attempts to have language models do both—generating code that performs numerical computation—conflate interpretation with execution and sacrifice the reliability that scientific work requires (Schick *et al.* 2023).

If parameter extraction improves—through fine-tuning, structured output constraints, or two-pass verification—the bottleneck shifts from “did the model understand the request?” to “does the method library cover this analysis?” At that point, extending coverage becomes more valuable than improving the language model, which is exactly the division of labor the architecture is designed to exploit.

The viability of local LLM deployment shown in Section 6 has implications beyond data privacy. If small models can reliably select among pre-implemented methods, conversational interfaces to scientific software become possible without cloud connectivity or powerful hardware. The design of **prompt2analytics** suggests a template: implement algorithms in high-performance, memory-safe languages; expose them through structured protocols; and let language models handle the translation between human intent and computational execution.

Acknowledgments

The authors thank the reviewers for constructive feedback that improved the paper.

Appendix

Contents

- A Best practices and coverage gaps
- B LLM evaluation details
- C Extended robustness evaluation
- D API response latency
- E Deployment guide
- F Privacy and security
- G Reproducibility
- H Benchmark methodology
- I Related work: Statistical interfaces
- J Implementation notes
- K Extended examples

A. Best practices and coverage gaps

This appendix provides practical guidance for effective use of **prompt2analytics** and documents method coverage gaps.

A.1. Pre-analysis checklist

Before running an analysis through **prompt2analytics**, users should verify:

1. *Method availability*: Confirm the required method is implemented. Table 8 in Section 5 provides feature coverage; Appendix J gives comprehensive method documentation.
2. *Data requirements*: Verify the dataset meets method assumptions. Panel methods require entity and time identifiers; IV requires instruments; DiD requires treatment and time indicators.
3. *Variable naming*: Use clear, unambiguous column names. The LLM extracts variable names from natural language; names like `x1` or `var_1` are more error-prone than `wage` or `education`.
4. *Specification clarity*: Specify methods explicitly when precision matters. “Estimate a within-transformation fixed effects model with clustered standard errors at the firm level” yields more reliable results than “run a regression with firm effects.”

A.2. Recommended workflows

The system is most effective for:

- *Exploratory analysis*: Quickly testing specifications before committing to a formal analysis pipeline.

- *Teaching and learning*: Students can explore econometric methods conversationally while seeing which tools and parameters are invoked.
- *Rapid prototyping*: Researchers can iterate on model specifications in natural language before writing production code.
- *Verification*: Cross-checking results from other software by running the same analysis through an independent implementation.

Users should prefer R, Stata, or Python when methods are unavailable (Bayesian inference, Heckman selection), maximum performance is critical (**fixest** outperforms for HDFE at scale), reproducibility requires code scripts, or publication-quality visualizations are needed.

A.3. Method coverage gaps

While method coverage is broad (200+ methods), several important categories remain unimplemented. Table A.1 summarizes coverage gaps and recommends alternative tools.

Table A.1: Method coverage gaps

Method Category	Status	Recommended	Alternative
Bayesian inference	Not implemented	R: rstanarm , Python: PyMC	brms ;
Heckman selection	Not implemented	R: sampleSelection ; Stata: heckman	
Conley spatial SEs	Not implemented	R: fixest , lfe	
Full SARIMA (arbitrary seasonal)	Partial support	R: forecast	
Quantile IV	Not implemented	R: quantreg	
Structural equation modeling	Not implemented	R: lavaan	
Weak instrument inference	F -stat only; no Stock-Yogo CVs	R: ivmodel , AER	

Note: Lists unimplemented or partially implemented method categories with recommended alternatives in R, Stata, or Python.

B. LLM evaluation details

This appendix provides detailed results from the tool selection evaluation described in Section 5. Table B.1 presents accuracy, cost, and failure rates across all evaluated models. Table B.2 breaks down accuracy by test category.

C. Extended robustness evaluation

This appendix provides detailed results from the extended evaluation described in Section 5. The evaluation assesses model performance across five dimensions beyond single-turn tool se-

Table B.1: Tool selection accuracy and cost by model

Model	Size	Provider	N	Exact	Fail	Acc. (%)	Cost (\$)
Claude 3.5 Haiku	—	Anthropic	87	87	0	100.0	2.65
Qwen 2.5 72B	72B	OpenRouter	87	87	0	100.0	2.63
Nemotron 9B [†]	9B	OpenRouter	43	43	0	100.0	0.22
Llama 3.3 70B	70B	OpenRouter	87	86	0	98.9	1.17
DeepSeek [†]	—	OpenRouter	65	64	1	98.5	1.11
GPT-4o Mini	—	OpenAI	87	85	1	97.7	1.47
GPT-5 Nano	—	OpenAI	87	85	1	97.7	4.91
GPT-4.1 Nano	—	OpenAI	87	84	2	96.6	0.98
Minstral 3B	3B	OpenRouter	87	79	2	90.8	0.39
Qwen3 8B [†]	8B	OpenRouter	26	24	2	88.5	0.13

Note: [†] = partial evaluation due to API rate limits; interpret with caution. Cost estimates based on $\sim 11,000$ input tokens (tool schema) + 75 tokens (prompt/response) at January 2026 pricing. Local deployment has zero marginal API cost.

Table B.2: Accuracy by test category (%)

Category	Haiku	Qwen	Llama	4o-mini	5-nano	4.1-nano	Mini-3B
Regression	100.0	100.0	100.0	100.0	100.0	90.0	90.0
Panel Data	100.0	100.0	90.0	100.0	100.0	100.0	100.0
Causal Inference	100.0	100.0	100.0	100.0	100.0	100.0	90.9
Discrete Choice	100.0	100.0	100.0	90.0	100.0	80.0	100.0
Time Series	100.0	100.0	100.0	100.0	100.0	100.0	66.7
Hypothesis Testing	100.0	100.0	100.0	91.7	91.7	100.0	83.3
Machine Learning	100.0	100.0	100.0	100.0	91.7	100.0	100.0
Visualization	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Overall	100.0	100.0	98.9	97.7	97.7	96.6	90.8

Note: Each category contains 8–13 test cases covering basic to advanced use. Bold row shows overall accuracy across all 87 tests.

lection: multi-turn conversations, naturalistic prompts, parameter extraction, interpretation accuracy, and out-of-scope detection.

C.1. Evaluation methodology

Multi-turn conversations. Twenty conversations with 3–4 turns each (72 total turns) simulate realistic analytical workflows where subsequent requests build on prior context. Each turn is scored independently; a conversation is “complete” only if all turns are correct.

Naturalistic prompts. Ninety-nine prompts across eight categories include realistic variations: typos (“regresion with robust se”), informal language (“just run a quick ols”), domain jargon (“estimate the ATT using DID”), verbose descriptions, and ambiguous specifications. Prompts are classified by variation type to identify specific robustness gaps.

Parameter extraction. Forty-nine test cases evaluate whether models correctly extract variable names, options (robust SE type, clustering variables), and modifiers from natural language. We compute precision (correct extractions / total extractions), recall (correct extractions / expected parameters), and F1 score.

Interpretation accuracy. Twenty-six statistical outputs spanning coefficient interpretation, hypothesis test results, and diagnostic statistics are presented to each model with interpretation prompts. Responses are scored for coverage of expected elements and absence of common errors (e.g., confusing correlation with causation, misinterpreting log coefficients).

Out-of-scope detection. Twenty prompts request methods outside the **prompt2analytics** scope (Bayesian inference, Heckman selection, structural equation modeling, etc.). Correct responses either decline the request or suggest the closest available alternative; incorrect responses select an inappropriate tool.

C.2. Results

Table C.1 summarizes performance. The main takeaway: multi-turn accuracy ranges from 47–97% with tool calls in history, and drops to 47–60% without. Context management explains most of the variance. Parameter extraction stays weak regardless (F1 of 41–59%).

Figure C.1 shows that no model wins everywhere. Claude 3.5 Haiku does well on single-turn and naturalistic prompts but not multi-turn. GPT-4o-mini achieves the highest multi-turn accuracy (97.2%). Qwen 2.5 72B extracts parameters most reliably. Pick based on how you plan to use it.

Figure C.2 compares controlled versus realistic conditions. With tool calls in history, GPT-4o-mini hits 97% multi-turn accuracy (versus 98% single-turn)—the gap nearly disappears. Without tool calls in history, all models drop 30–40 percentage points. Context management, not model size, determines multi-turn performance.

C.3. Failure analysis

Common failure patterns across dimensions include:

- *Context drift:* Without tool calls in history, models “forget” prior specifications and misread pronoun references. Including tool calls in history fixes this.

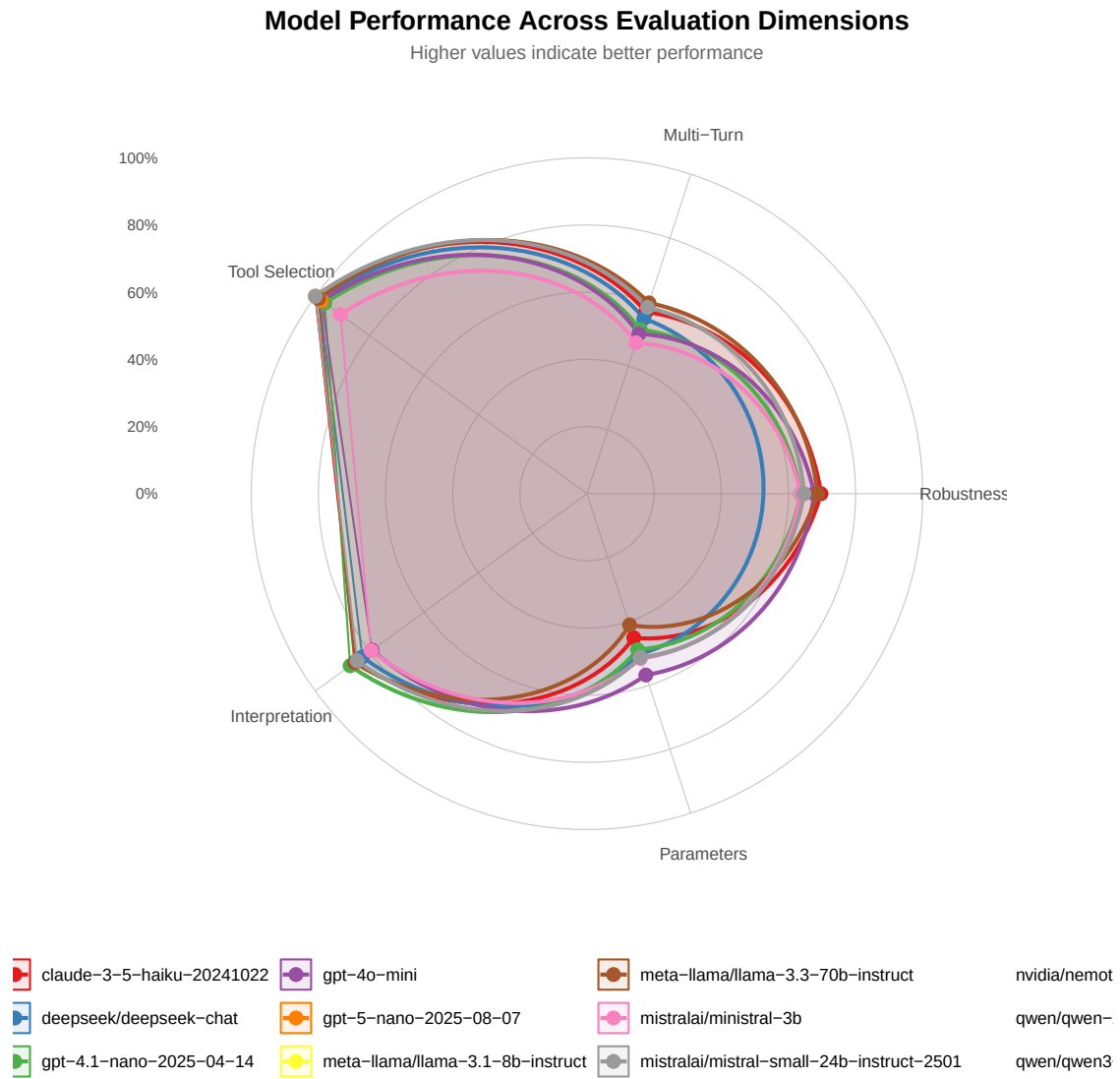


Figure C.1: Performance profile by model across robustness dimensions (normalized to 0–100%). Models cluster in the 50–70% range; no single model dominates all dimensions.

Extended LLM Tool Selection Evaluation

Comparing performance across evaluation types

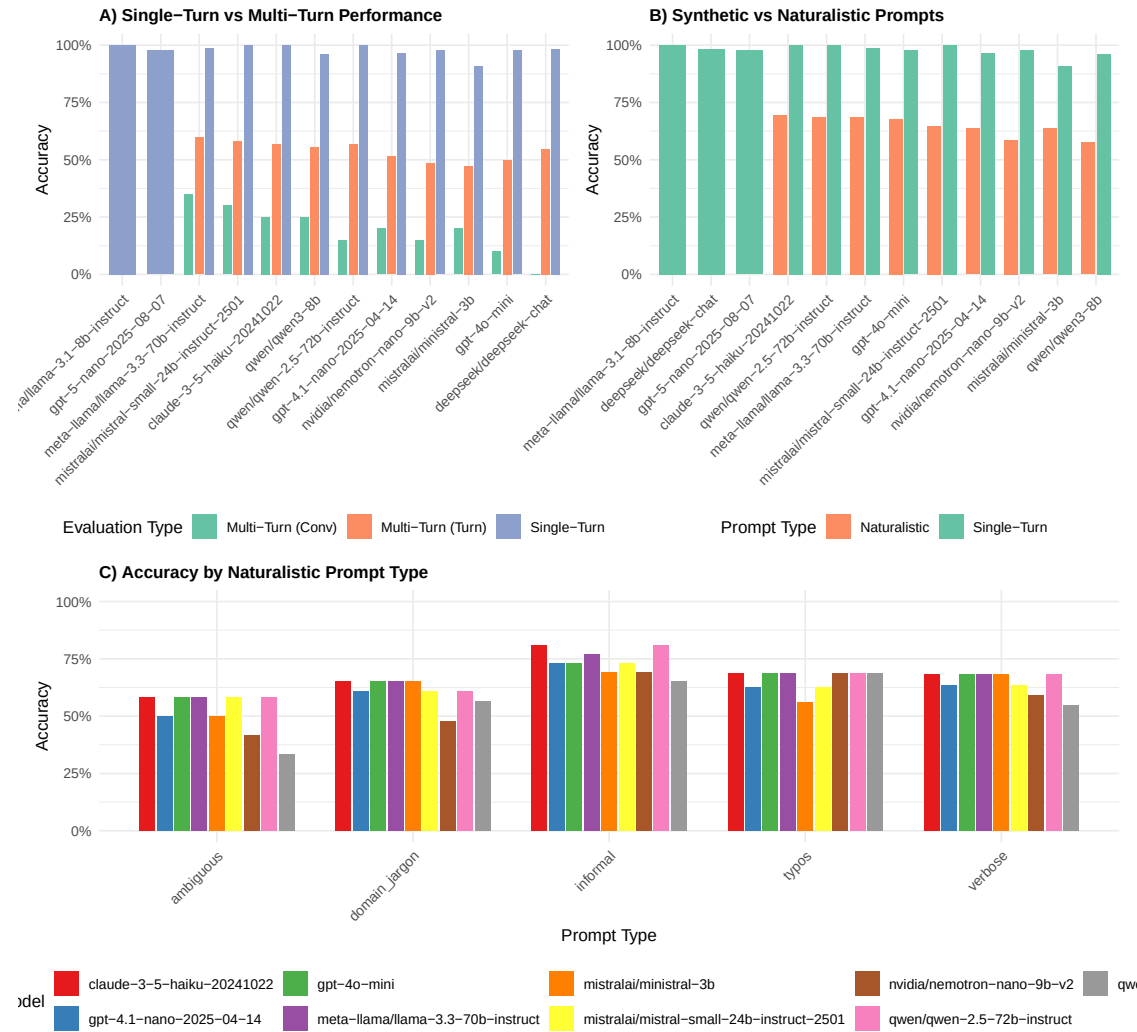


Figure C.2: Controlled vs. realistic accuracy. Left: single-turn vs. multi-turn. Right: synthetic vs. naturalistic prompts. The gap shows that baseline evaluations overstate practical capability.

Table C.1: Extended evaluation: per-model results across robustness dimensions

Model	Single-Turn	Multi-Turn	Conv. Compl.	Naturalistic	Param F1	Interpret.	OOS Det.
GPT-4o-mini	97.7	97.2	90.0	67.7	56.8	75.6	55.0
Claude 3.5 Haiku	100.0	56.9	25.0	69.7	45.1	85.3	45.0
GPT-4.1 Nano	96.6	51.4	20.0	63.6	48.8	87.2	50.0
Qwen3 8B	96.2	83.3	60.0	57.6	43.7	67.8	55.0
Qwen 2.5 7B	—	66.7	50.0	—	—	—	—
Llama 3.3 70B	98.9	59.7	35.0	68.7	41.1	85.6	55.0
Qwen 2.5 72B	100.0	56.9	15.0	68.7	59.1	82.7	—
Mistral Small 24B	100.0	58.3	30.0	64.6	51.2	84.6	55.0
Ministral 3B	90.8	47.2	20.0	63.6	51.6	79.5	40.0
Nemotron Nano 9B	97.7	48.6	15.0	58.6	51.9	80.3	60.0

Note: Single-Turn = tool selection accuracy (87 tests); Multi-Turn = per-turn accuracy (72 turns); Conv. Compl. = all turns correct (20 conversations); Naturalistic = informal prompts (99 tests); Param F1 = parameter extraction (49 tests); Interpret. = interpretation (26 tests); OOS Det. = out-of-scope detection (20 tests). Multi-turn uses enhanced context (tool calls in history). Dashes = incomplete data due to API errors.

- *Partial parameter extraction:* Models often extract the primary parameters (dependent/independent variables) but omit secondary options (robust SE type, clustering specification, interaction terms).
- *Overconfident tool selection:* When facing out-of-scope requests, models rarely decline; instead, they select superficially similar tools (e.g., frequentist regression for Bayesian regression, standard VAR for Bayesian VAR).
- *Interpretation shortcuts:* Models sometimes provide accurate but incomplete interpretations, omitting important caveats about statistical vs. practical significance or the limitations of specific tests.

These patterns inform the deployment recommendations in Section 6: explicit specifications reduce context drift, parameter verification catches extraction errors, and result review identifies interpretation gaps.

D. API response latency

Figure D.1 presents response latency distributions for the models evaluated via their respective cloud APIs. These measurements include network round-trip time and reflect API-hosted inference, not local deployment performance.

For cloud-hosted inference, GPT-4.1 Nano shows the fastest median response (1,245ms) among high-accuracy models, while Ministral 3B achieves 650ms median latency via OpenRouter. These figures are useful for comparing relative API performance but do not predict local inference speed.

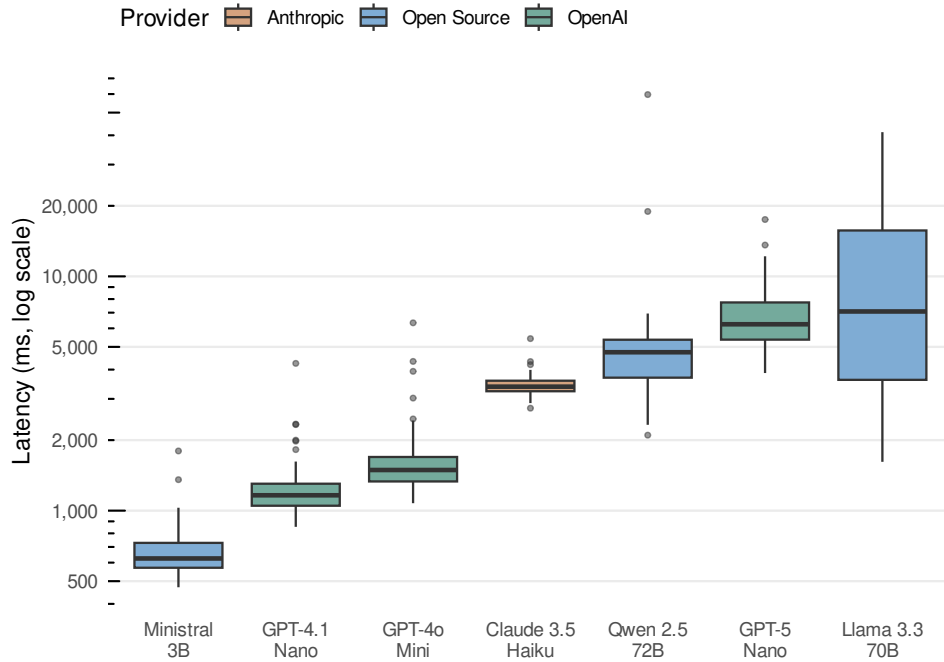


Figure D.1: API response latency by model (log scale). Latencies reflect cloud-hosted inference including network round-trip time; local deployment would differ substantially.

Local deployment latency depends on several factors: model size and quantization level, available RAM and GPU VRAM, CPU or GPU compute capability, and context length (which affects KV cache size). As a rough guide, GPU-accelerated inference on consumer hardware (RTX 3070/4070 with 8 GB VRAM) typically achieves 1–3 seconds for 7–8B parameter models. CPU-only inference on modern laptops ranges from 5–15 seconds depending on model size and system specifications.

E. Deployment guide

This appendix provides deployment instructions for various platforms.

E.1. Linux workstation

1. Install Rust (1.85+): `curl -proto '=https' -tlsv1.2 -sSf https://sh.rustup.rs | sh`
2. Clone repository:
`git clone https://github.com/prompt2analytics/prompt2analytics`
3. Build MCP server: `cargo build -release -p p2a-mcp`
4. Install Ollama: `curl -fsSL https://ollama.ai/install.sh | sh`

5. Pull a model: `ollama pull llama3.1:8b`
6. Run MCP server: `./target/release/p2a-mcp -transport stdio`
7. Configure Claude Desktop or other MCP client to use the server

E.2. macOS (Apple Silicon)

Apple M-series chips provide excellent performance for local LLM inference due to unified memory architecture. Recommended configuration:

- M1/M2: 7–8B models with Q4_K_M quantization
- M2 Pro/Max: 13B models or 8B at Q8 quantization
- M3 Max/Ultra: Up to 70B models with sufficient unified memory

Build commands are identical to Linux. Homebrew can simplify dependency management: `brew install rust ollama`.

E.3. Windows (WSL2)

Native Windows builds are supported but WSL2 provides a more consistent experience:

1. Install WSL2: `wsl -install -d Ubuntu`
2. Follow Linux instructions within WSL2
3. For GPU acceleration, install NVIDIA CUDA drivers on Windows (WSL2 will use them automatically)

E.4. MCP client configuration

For Claude Desktop, add to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "prompt2analytics": {
      "command": "/path/to/p2a-mcp",
      "args": ["--transport", "stdio"]
    }
  }
}
```

E.5. Pre-compiled binaries

For users who prefer not to install the Rust toolchain, pre-compiled binaries are available for common platforms:

- **Linux (x86_64-unknown-linux-gnu)**: Statically linked binary, no runtime dependencies except glibc 2.17+
- **macOS (aarch64-apple-darwin)**: Universal binary for Apple Silicon
- **macOS (x86_64-apple-darwin)**: Intel Mac binary
- **Windows (x86_64-pc-windows-msvc)**: MSVC-linked binary

Download from the GitHub releases page:

<https://github.com/umatter/prompt2analytics/releases>

After downloading, extract and run directly:

```
# Linux/macOS
chmod +x p2a-mcp
./p2a-mcp --transport stdio

# Windows
p2a-mcp.exe --transport stdio
```

E.6. Docker deployment

For isolated, reproducible deployment without any local installation:

```
# Pull the official image
docker pull ghcr.io/umatter/prompt2analytics:latest

# Run with local data directory mounted
docker run -v /path/to/data:/data \
  -p 8080:8080 \
  ghcr.io/umatter/prompt2analytics:latest \
  --transport http --host 0.0.0.0 --port 8080

# Or use docker-compose for persistent deployment
docker-compose up -d
```

The Docker image includes:

- Pre-built p2a-mcp and p2a binaries
- Ollama with Llama 3.1 8B pre-downloaded
- All BLAS dependencies (OpenBLAS)
- Example datasets for testing

E.7. Model selection guidance

See Tables 12 and 13 in Section 6 for detailed model recommendations by use case and hardware requirements.

E.8. Hardware requirements for data processing

Beyond LLM memory, users must budget RAM for dataset operations. Rust processes data in compact columnar format via **polars**. The primary memory consumers are design matrices ($8nk$ bytes for n observations and k predictors), covariance matrices ($8k^2$ bytes), bootstrap samples ($8kB$ bytes for B replications), and panel transformations (approximately equal in size to the original dataset).

Table E.1: RAM requirements for data processing

Rows	File size	Base RAM	Matrix ops	Notes
1,000	<1 MB	50 MB	+20 MB	Standard laptop
10,000	1–10 MB	100 MB	+100 MB	Standard laptop
100,000	10–100 MB	500 MB	+500 MB	Consumer hardware
1,000,000	100 MB–1 GB	2 GB	+2 GB	Workstation
10,000,000	1–10 GB	10 GB	+8 GB	High-memory system

Note: Peak memory during analysis. Assumes typical econometric datasets with 10–50 columns.

For combined LLM and data processing, sum the requirements from Tables 13 and E.1. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM. Table E.2 provides system recommendations for common deployment scenarios.

Table E.2: System configurations by deployment scenario

Scenario	Model	RAM	GPU VRAM	Storage
Minimal (laptop)	Minstral 3B	16 GB	6 GB	5 GB
Consumer (desktop)	Qwen 2.5 7B	16 GB	8 GB	10 GB
Professional	Llama 3.3 70B	64 GB	48 GB	50 GB
Server (large data)	Qwen 2.5 72B	128 GB	2×24 GB	60 GB

Note: RAM includes both LLM and data processing headroom. GPU requirements assume CUDA-capable NVIDIA cards; AMD and Intel GPUs are supported via Ollama’s ROCm and OneAPI backends.

F. Privacy and security

This appendix documents data transmission behavior and security considerations.

F.1. Data transmission summary

Table F.1 lists operations that may transmit data beyond the local machine when using cloud LLM providers.

Table F.1: Data transmission by operation type

Operation	Data Transmitted	Risk Level
Dataset load	Column names, types, row count, null percentages	Low
Data preview (head/tail/sample)	Actual row values	High
Data profiling	Frequent values, min/max, examples	Medium
Statistical estimation	Coefficients, SEs, test statistics only	Low
Outlier detection	Specific observation values	Medium
Error messages	May include problematic values	Low

Note: Applies only when using cloud LLM providers. Local deployment via Ollama eliminates all external data transmission.

F.2. Audit logging

To enable audit logging of all MCP tool calls:

```
p2a-mcp --transport stdio --log-level debug --log-file audit.log
```

The log records: timestamp, tool name, parameters (excluding bulk data), and result summary. Review logs to verify no unexpected data transmission.

F.3. Network verification

To verify local-only operation with Ollama:

1. Start Ollama: `ollama serve`
2. Block outbound connections: `sudo iptables -A OUTPUT -j DROP`
3. Run analysis—should complete successfully
4. Restore network: `sudo iptables -D OUTPUT -j DROP`

F.4. Security best practices

- Avoid loading untrusted data files (potential injection payloads)
- Review tool calls when processing sensitive data
- Use local LLM deployment for high-security environments
- Keep software updated; security patches are released as needed
- The MCP server has no filesystem access beyond loaded datasets

G. Reproducibility

This appendix provides detailed instructions for reproducing the results in this paper.

G.1. Version information

Results in this paper were generated with the following software versions:

- Rust: 1.85.0 (stable)
- R: 4.5.0 with OpenBLAS
- Key R packages: **plm** 2.6-4, **lfe** 3.1-0, **sandwich** 3.1-1, **fixest** 0.12.1, **forecast** 8.23.0
- Operating system: Ubuntu 24.04 LTS (Linux 6.8)

G.2. Reproducing benchmarks

Performance benchmarks can be reproduced using the following protocol:

1. Install R 4.5+ with required packages: **plm**, **lfe**, **sandwich**, **forecast**, **microbenchmark**
2. Build **prompt2analytics** in release mode: `cargo build -release -p p2a-cli`
3. Execute benchmarks: `make -C paper/code benchmarks`

Both languages use `seed = 42` for random number generation. The benchmark protocol includes 10 warmup iterations before 100 measured iterations for fast methods (50 for moderate, 20 for slow). Detailed instructions: `paper/code/README_REPRODUCIBILITY.md`.

G.3. Containerized reproduction

A `Dockerfile` in the repository root provides a containerized environment with all dependencies pinned to the versions above:

```
docker build -t p2a-paper .
docker run -v $(pwd)/paper:/output p2a-paper make all
```

The container includes both R and Rust toolchains with identical configurations to those used for this paper.

H. Benchmark methodology

This appendix provides complete details on the benchmark methodology used in Section 6.

H.1. Data generating processes

Benchmark datasets are generated synthetically using identical data-generating processes in both languages with fixed random seeds (`seed = 42`) for exact reproducibility.

Regression benchmarks. Data follow $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i$ with true parameters $\beta_0 = 1.0$, $\beta_1 = 2.0$, $\beta_2 = -1.5$, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ where $\sigma = 1.0$. Covariates are drawn as $x_{1i} \sim \mathcal{N}(0, 1)$ and $x_{2i} \sim \mathcal{N}(0, 1)$ with correlation $\rho = 0.3$.

Panel data benchmarks. Entity effects $\alpha_i \sim \mathcal{N}(0, 0.5^2)$ are added, with 100 entities observed over 10 periods.

Discrete choice benchmarks. The latent index $y_i^* = \beta'x_i + \varepsilon_i$ is transformed via the logit or probit link.

Time series benchmarks. Data follow $\text{ARIMA}(1, 1, 1)$ with $\phi = 0.7$, $\theta = 0.3$, and innovation variance $\sigma_\eta^2 = 1.0$.

Complete DGP specifications are provided in `paper/code/README_REPRODUCIBILITY.md`.

H.2. Benchmark execution protocol

Each benchmark configuration executes multiple iterations after a warmup phase:

- 100 iterations for fast methods (OLS, robust SEs, PCA)
- 50 iterations for moderate methods (panel FE, clustering)
- 20 iterations for slow methods (ARIMA, MSTL decomposition)

R benchmarks use the **bench** package, which handles garbage collection timing. Rust benchmarks use Criterion with automatic warmup and outlier detection. We report full timing distributions (minimum, quartiles, maximum, mean, standard deviation) rather than point estimates alone.

H.3. Hardware and software environment

All benchmarks run on identical hardware: an Intel Core i7-1260P processor with 64GB RAM running Linux, with R 4.5.2 using OpenBLAS and Rust 1.92.0 compiled in release mode with link-time optimization.

H.4. Validation tolerances

Numerical validation against R reference implementations uses sample-size-dependent tolerances. We use absolute tolerances for quantities with known scale (e.g., test statistics) and relative tolerances for unbounded quantities (e.g., coefficient estimates).

Rationale for tolerances. Looser tolerances for small samples reflect increased sensitivity to algorithmic differences when matrices are ill-conditioned. HC2/HC3 standard errors use looser tolerances because leverage calculation involves $h_{ii} = x_i'(X'X)^{-1}x_i$, which accumulates rounding errors differently depending on whether leverage is extracted from QR decomposition (as in **sandwich**) or computed via direct matrix multiplication (as in **prompt2analytics**). Both approaches are numerically valid; the tolerance reflects implementation variation rather than error.

MLE methods use looser small-sample tolerances because convergence criteria differ between implementations: R's `glm()` uses relative change in deviance, while **prompt2analytics** uses absolute change in log-likelihood. Final parameter estimates can differ at the tolerance boundary while both satisfying their respective convergence criteria.

Table H.1: Validation tolerances by method and sample size

Category	Quantity	Small ($n < 100$)	Medium	Large ($n > 10,000$)
<i>Coefficients and standard errors</i>				
OLS coefficients	Relative	10^{-6}	10^{-8}	10^{-10}
Robust SEs (HC0/HC1)	Relative	10^{-6}	10^{-8}	10^{-8}
Robust SEs (HC2/HC3)	Relative	10^{-5}	10^{-7}	10^{-7}
Panel FE/RE	Relative	10^{-5}	10^{-6}	10^{-6}
MLE (logit/probit)	Relative	10^{-4}	10^{-6}	10^{-6}
<i>Test statistics and fit measures</i>				
t -statistics, F -statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
R^2 , within R^2	Absolute	10^{-4}	10^{-6}	10^{-6}
p -values	Absolute	10^{-4}	10^{-6}	10^{-6}
Log-likelihood	Absolute	10^{-2}	10^{-4}	10^{-4}
<i>Hypothesis tests</i>				
Test statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
Critical values	Absolute	10^{-4}	10^{-4}	10^{-4}

Note: Absolute tolerances (ε_{abs}) apply to test statistics, p -values, and bounded quantities. Relative tolerances (ε_{rel}) apply to coefficient estimates and standard errors.

Tolerance verification. Tolerances were determined empirically by running validation tests across sample sizes from $n = 10$ to $n = 100,000$ and identifying the tightest tolerance that passes consistently. Edge cases (singular matrices, near-separation in logit/probit, small cluster counts) are tested separately with method-specific tolerances documented in the test code. All 334 validation tests pass at the specified tolerances.

I. Related work: Statistical interfaces

This appendix provides extended discussion of related work on statistical computing interfaces.

I.1. Formula notation and domain-specific languages

The most influential interface innovation in statistical computing was the Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973), which introduced the symbolic representation of linear models that remains central to R, Stata, and other systems (e.g., $y \sim x1 + x2$). Chambers and Hastie (1992) extended this notation in S and later R, establishing design principles for statistical software that prioritize expressiveness and composability (Chambers 1998, 2008). The formula interface represents a domain-specific language (DSL) for model specification—a pattern that Stata adopts with its command syntax and that appears in specialized systems like Stan’s probabilistic programming language (Carpenter, Gelman, Hoffman, Lee, Goodrich, Betancourt, Brubaker, Guo, Li, and Riddell 2017).

prompt2analytics takes a fundamentally different approach: rather than inventing new notation, it accepts natural language descriptions of analytical intent. This trades the precision of formal DSLs for accessibility, relying on LLMs to bridge the gap between informal specification and structured execution.

I.2. Natural language interfaces to data systems

Natural language interfaces (NLIs) for data systems have a long history. LUNAR (Woods 1973) demonstrated feasibility for database querying; subsequent decades produced various NLI systems with limited adoption (Androutsopoulos, Ritchie, and Thanisch 1995). Commercial systems attempted similar approaches: SPSS’s syntax assistance, SAS Enterprise Guide’s point-and-click interface with natural language elements, and JMP’s scripting language with English-like constructs.

The text-to-SQL paradigm, revitalized by deep learning, represents recent progress on natural language data interfaces. Benchmarks like WikiSQL (Zhong, Xiong, and Socher 2017) and Spider (Yu *et al.* 2018) have driven substantial improvements in translating natural language to structured queries (Li *et al.* 2024). NL4DV (Narechania, Srinivasan, and Stasko 2021) enables natural language specification of visualizations.

I.3. Automated machine learning

AutoML systems automate model selection through algorithmic search. Auto-WEKA (Thornton, Hutter, Hoos, and Leyton-Brown 2013), Auto-sklearn (Feurer *et al.* 2015), and H2O AutoML (LeDell and Poirier 2020) optimize predictive performance over predefined pipeline spaces. Recent work integrates LLMs into AutoML: Hollmann, Müller, and Hutter (2024) use language models to guide feature engineering and hyperparameter search.

AutoML differs from chat-first analytics in objective and scope. AutoML optimizes for prediction; chat-first analytics interprets user intent across prediction, causal inference, hypothesis testing, and description.

I.4. Computational econometrics

The computational econometrics literature has established standards for numerical accuracy and implementation quality. McCullough (1997) and McCullough and Vinod (1999) documented numerical inaccuracies across commercial and open-source statistical packages. Textbooks by Davidson and MacKinnon (2004) and Cameron and Trivedi (2005) provide authoritative treatments of econometric methods and their implementation.

J. Implementation notes

This appendix documents implementation caveats and limitations for specific methods.

J.1. Instrumental variables

The 2SLS implementation reports first-stage F -statistics for instrument relevance. The traditional rule-of-thumb ($F > 10$) from Staiger and Stock (1997) has been superseded by more nuanced guidance. Stock and Yogo (2005) provide critical values depending on the number of instruments and acceptable bias levels; Lee, McCrary, Moreira, and Porter (2022) offer further refinements for inference under weak identification. The current implementation does not provide Stock-Yogo critical values or weak-identification-robust inference methods (Anderson-Rubin, conditional likelihood ratio). Users with potentially weak instruments should verify results using R’s **ivmodel** or **AER** packages.

J.2. Difference-in-differences

The DiD implementation assumes the canonical two-group, two-period design. Recent econometric literature has documented severe biases when this estimator is applied to staggered treatment timing. [Goodman-Bacon \(2021\)](#) demonstrates that two-way fixed effects DiD produces a weighted average of all possible 2×2 comparisons, including problematic “forbidden” comparisons using already-treated units as controls. [de Chaisemartin and D’Haultfoeuille \(2020\)](#) and [Sun and Abraham \(2021\)](#) propose alternative estimators robust to treatment effect heterogeneity.

prompt2analytics implements staggered DiD via Callaway-Sant’Anna ([Callaway and Sant’Anna 2021](#)), Goodman-Bacon decomposition, and extended TWFE ([Wooldridge 2021](#)). Users with complex staggered designs may also consult **fixest**’s `sunab()` function or the **did** package in R for additional diagnostics.

J.3. Random effects

The random effects estimator uses the Swamy-Arora variance component estimator, which can produce negative variance estimates for σ_α^2 (the between-entity variance) in certain data configurations. When this occurs, the implementation truncates the estimate to zero and proceeds with pooled OLS, issuing a warning.

J.4. Discrete choice models

Logit and probit models report coefficients on the latent index scale. For interpretation, marginal effects are computed as *Average Marginal Effects* (AME): the partial derivative $\partial \Pr(y = 1) / \partial x_j$ is evaluated at each observation’s covariate values, then averaged across the sample. This differs from *Marginal Effects at Means* (MEM), which evaluates derivatives at the sample mean of all covariates. AME is generally preferred because it does not depend on potentially unrepresentative “average” covariate values ([Cameron and Trivedi 2005](#)).

J.5. Time series models

ARIMA estimation supports non-seasonal models: $\text{ARIMA}(p, d, q)$ with automatic or user-specified orders. Seasonal ARIMA (SARIMA) is partially implemented: seasonal differencing and seasonal AR/MA terms can be specified, but automatic seasonal order selection is not available.

VAR and VECM implementations support lag selection via AIC/BIC and Johansen cointegration testing, but do not implement structural VAR identification.

J.6. Regression diagnostics validation

Diagnostic tests are validated against **lmtest** ([Hothorn, Zeileis, Farebrother, and Cummins 2024](#)): Jarque-Bera normality test statistics match to $< 10^{-4}$, Breusch-Pagan heteroskedasticity tests to $< 10^{-4}$, Durbin-Watson autocorrelation statistics to $< 10^{-6}$, and variance inflation factors (VIF) to $< 10^{-6}$.

J.7. Known differences from reference implementations

A small number of intentional differences exist between **prompt2analytics** and reference implementations. First, **prompt2analytics** defaults to HC1 robust standard errors (consistent with Stata’s **robust** option), while R’s **sandwich** defaults to HC3. Second, R uses variable names in output; **prompt2analytics** uses numeric indices internally but displays names in formatted output. Third, for panel models we report “within R^2 ” by default, matching **plm**’s convention for fixed effects models. Fourth, iterative methods (HDFE, MLE) use a default convergence tolerance of 10^{-8} , which may differ slightly from R package defaults. These differences are documented in the method documentation and do not affect the validity of statistical inference.

J.8. Methods not implemented

The current implementation is comprehensive, covering 200+ methods. Remaining unimplemented categories include: sample selection (Heckman) and Bayesian methods. Users requiring these should use R (e.g., **sampleSelection**, **rstanarm**) or Stata.

K. Extended examples

This appendix provides additional worked examples demonstrating **prompt2analytics** capabilities.

K.1. Multi-step workflow example

Real-world analysis typically involves multiple steps beyond single-command estimation. This section demonstrates a realistic workflow including data inspection, handling issues, and specification refinement.

Initial exploration

User: I have employee salary data in salaries.csv. Load it and tell me what variables are available for analysis.

****Dataset loaded: salaries****

- Rows: 1,847

- Columns: 8

Column	Type	Non-null	Description
employee_id	Int64	100%	Unique identifier
salary	Float64	98.2%	Annual salary (USD)
education	String	99.5%	Highest degree (HS/BA/MA/PhD)
experience	Float64	97.8%	Years of experience
department	String	100%	Department name
gender	String	100%	M/F
performance	Float64	94.1%	Performance rating (1-5)
tenure	Int64	100%	Years at company

I notice some missing values, particularly in performance (5.9% missing) and experience (2.2% missing). Would you like me to examine the missingness pattern before proceeding?

Addressing data issues

User: Yes, check if the missing values are random or if there's a pattern. Then proceed with complete cases for now.

****Missingness Analysis****

Performance ratings are more likely to be missing for:

- Recent hires (tenure < 1 year): 23% missing vs 3% for others
- This suggests MNAR (missing not at random) - new employees may not yet have performance reviews.

****Proceeding with complete cases:**** 1,692 observations (91.6% retained)

Initial specification

User: Regress salary on education, experience, and gender. I want to understand the gender pay gap controlling for human capital.

****OLS Regression Results**** (HC1 robust standard errors)

Variable	Coef	Std Err	t	P> t
(Intercept)	42,156	1,247	33.81	0.000
education_BA	8,934	892	10.02	0.000
education_MA	15,267	1,103	13.84	0.000
education_PhD	22,891	1,456	15.72	0.000
experience	1,247	89	14.01	0.000
gender_M	4,892	743	6.58	0.000

R-squared: 0.412

Specification refinement

User: Add department fixed effects and performance rating.

****Regression with Department Fixed Effects****

Variable	Coef	Std Err	t	P> t
education_BA	6,234	812	7.68	0.000
education_MA	11,456	1,021	11.22	0.000
education_PhD	17,234	1,298	13.28	0.000
experience	1,089	82	13.28	0.000
performance	3,456	312	11.08	0.000
gender_M	2,134	698	3.06	0.002

R-squared (within): 0.523

Department FE: 12 groups

Adding department controls and performance reduces the gender coefficient from \\$4,892 to \\$2,134 (a 56% reduction).

K.2. Additional CLI examples

The command-line interface provides programmatic access for scripted workflows:

```
# Session-based workflow
p2a --session analysis.json data load grunfeld.csv --name grunfeld
p2a --session analysis.json data describe grunfeld

# Panel data estimation with clustering
p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm

# Export session to reproducible script
p2a script export analysis.json -o reproduce.sh
```

K.3. Additional error handling examples

Small cluster warnings

User: Cluster the standard errors by department.

****Clustered Standard Errors (by department)****

Variable	Coef	Robust SE	Cluster SE	t	P> t
performance	0.142	0.018	0.031	4.58	0.001
tenure	0.023	0.004	0.007	3.29	0.008
experience	0.008	0.003	0.004	2.00	0.071

****Warning:** Small number of clusters (G = 10)**

Clustered standard errors require $G \rightarrow \infty$ asymptotics. With only 10 clusters, standard errors may be biased downward.

Out-of-scope requests

User: Run a Bayesian regression with informative priors.

****Method not available****

Bayesian inference is not currently implemented. Available alternatives:

- OLS with robust standard errors (implemented)
- Ridge/LASSO regularization for coefficient shrinkage (implemented)
- Bootstrap confidence intervals (implemented)

****For Bayesian analysis**, consider:** R (rstanarm, brms), Python (PyMC).

References

- Abadie A, Diamond A, Hainmueller J (2010). “Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California’s Tobacco Control Program.” *Journal of the American Statistical Association*, **105**(490), 493–505. doi:[10.1198/jasa.2009.ap08746](https://doi.org/10.1198/jasa.2009.ap08746).
- Androutsopoulos I, Ritchie GD, Thanisch P (1995). “Natural Language Interfaces to Databases – An Introduction.” *Natural Language Engineering*, **1**(1), 29–81. doi:[10.1017/S135132490000005X](https://doi.org/10.1017/S135132490000005X).
- Anselin L (1988). *Spatial Econometrics: Methods and Models*. Springer, Dordrecht. doi:[10.1007/978-94-015-7799-1](https://doi.org/10.1007/978-94-015-7799-1).
- Anthropic (2024). “Model Context Protocol.” Accessed: 2024, URL <https://modelcontextprotocol.io/>.
- Arellano M, Bond S (1991). “Some Tests of Specification for Panel Data: Monte Carlo Evidence and an Application to Employment Equations.” *Review of Economic Studies*, **58**(2), 277–297. doi:[10.2307/2297968](https://doi.org/10.2307/2297968).
- Austin PC (2011). “An Introduction to Propensity Score Methods for Reducing the Effects of Confounding in Observational Studies.” *Multivariate Behavioral Research*, **46**(3), 399–424. doi:[10.1080/00273171.2011.568786](https://doi.org/10.1080/00273171.2011.568786).
- Baltagi BH (2013). *Econometric Analysis of Panel Data*. 5th edition. Wiley, Chichester.

- Bergé L (2018). “Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package **fixest**.” *CREA Discussion Papers*, **13**. URL <https://CRAN.R-project.org/package=fixest>.
- Callaway B, Sant’Anna PHC (2021). “Difference-in-Differences with Multiple Time Periods.” *Journal of Econometrics*, **225**(2), 200–230. doi:[10.1016/j.jeconom.2020.12.001](https://doi.org/10.1016/j.jeconom.2020.12.001).
- Calonico S, Cattaneo MD, Titiunik R (2014). “Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs.” *Econometrica*, **82**(6), 2295–2326. doi:[10.3982/ECTA11757](https://doi.org/10.3982/ECTA11757).
- Cameron AC, Trivedi PK (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press, Cambridge. doi:[10.1017/CB09780511811241](https://doi.org/10.1017/CB09780511811241).
- Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A (2017). “Stan: A Probabilistic Programming Language.” *Journal of Statistical Software*, **76**(1), 1–32. doi:[10.18637/jss.v076.i01](https://doi.org/10.18637/jss.v076.i01).
- Cattaneo MD, Feng Y, Titiunik R (2021). “Prediction Intervals for Synthetic Control Methods.” *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:[10.1080/01621459.2021.1979561](https://doi.org/10.1080/01621459.2021.1979561).
- Chambers JM (1998). *Programming with Data: A Guide to the S Language*. Springer-Verlag, New York. doi:[10.1007/978-1-4684-6310-8](https://doi.org/10.1007/978-1-4684-6310-8).
- Chambers JM (2008). *Software for Data Analysis: Programming with R*. Springer-Verlag, New York. doi:[10.1007/978-0-387-75936-4](https://doi.org/10.1007/978-0-387-75936-4).
- Chambers JM, Hastie TJ (1992). *Statistical Models in S*. Wadsworth & Brooks/Cole, Pacific Grove, CA.
- Chen S, Shen K, Ma X (2025). “Econometrics AI Agent.” *arXiv preprint arXiv:2506.00856*. URL <https://arxiv.org/abs/2506.00856>.
- Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). “Double/Debiased Machine Learning for Treatment and Structural Parameters.” *Econometrics Journal*, **21**(1), C1–C68. doi:[10.1111/ectj.12097](https://doi.org/10.1111/ectj.12097).
- Cinelli C, Hazlett C (2020). “Making Sense of Sensitivity: Extending Omitted Variable Bias.” *Journal of the Royal Statistical Society: Series B*, **82**(1), 39–67. doi:[10.1111/rssb.12348](https://doi.org/10.1111/rssb.12348).
- Codd EF (1974). “Seven Steps to Rendezvous with the Casual User.” *IFIP Working Conference on Data Base Management*, pp. 179–200.
- Croissant Y, Milla G (2008). “Panel Data Econometrics in R: The **plm** Package.” *Journal of Statistical Software*, **27**(2), 1–43. doi:[10.18637/jss.v027.i02](https://doi.org/10.18637/jss.v027.i02).
- Davidson R, MacKinnon JG (2004). *Econometric Theory and Methods*. Oxford University Press, New York.
- de Chaisemartin C, D’Haultfoeuille X (2020). “Two-Way Fixed Effects Estimators with Heterogeneous Treatment Effects.” *American Economic Review*, **110**(9), 2964–2996. doi:[10.1257/aer.20181169](https://doi.org/10.1257/aer.20181169).

- Feurer M, Klein A, Eggenberger K, Springenberg JT, Blum M, Hutter F (2015). “Efficient and Robust Automated Machine Learning.” In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2962–2970.
- Gaure S (2013). “**lfe**: Linear Group Fixed Effects.” *The R Journal*, **5**(2), 104–117. URL <https://journal.r-project.org/archive/2013/RJ-2013-031/>.
- Goodman-Bacon A (2021). “Difference-in-Differences with Variation in Treatment Timing.” *Journal of Econometrics*, **225**(2), 254–277. doi:10.1016/j.jeconom.2021.03.014.
- Google (2025). “Agent2Agent Protocol (A2A).” Announced April 2025; transferred to Linux Foundation, URL <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>.
- Greene WH (2018). *Econometric Analysis*. 8th edition. Pearson, New York.
- Hamilton JD (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.
- Hansen LP (1982). “Large Sample Properties of Generalized Method of Moments Estimators.” *Econometrica*, **50**(4), 1029–1054. doi:10.2307/1912775.
- Ho DE, Imai K, King G, Stuart EA (2011). “**MatchIt**: Nonparametric Preprocessing for Parametric Causal Inference.” *Journal of Statistical Software*, **42**(8), 1–28. doi:10.18637/jss.v042.i08.
- Hollmann N, Müller S, Hutter F (2024). “Large Language Models for Automated Machine Learning.” In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS)*. URL <https://arxiv.org/abs/2408.14033>.
- Hong S, Lin Y, Liu B, Liu B, Wu B, Li D, Chen J, Zhang J, Wang J, Zhang L, Zhang L, Yang M, Zhuge M, Guo T, Zhou T, Tao W, Wang W, Tang X, Lu X, Zheng X, Liang X, Fei Y, Cheng Y, Xu Z, Wu C (2025). “Data Interpreter: An LLM Agent For Data Science.” In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821. URL <https://arxiv.org/abs/2402.18679>.
- Hothorn T, Zeileis A, Farebrother RW, Cummins C (2024). **lmtest**: *Testing Linear Regression Models*. R package version 0.9-40, URL <https://CRAN.R-project.org/package=lmtest>.
- IBM Research (2025). “Agent Communication Protocol (ACP).” Launched March 2025; merged with A2A under Linux Foundation, URL <https://research.ibm.com/projects/agent-communication-protocol>.
- Imai K, Ratkovic M (2014). “Covariate Balancing Propensity Score.” *Journal of the Royal Statistical Society: Series B*, **76**(1), 243–263. doi:10.1111/rssb.12027.
- Koenker R, Jr GB (1978). “Regression Quantiles.” *Econometrica*, **46**(1), 33–50. doi:10.2307/1913643.
- Korinek A (2025). “AI Agents for Economic Research.” *Journal of Economic Literature*. Forthcoming, URL <https://www.aeaweb.org/content/file?id=23290>.

- LeDell E, Poirier S (2020). “H2O AutoML: Scalable Automatic Machine Learning.” In *Proceedings of the 7th ICML Workshop on Automated Machine Learning*. URL https://www.automl.org/wp-content/uploads/2020/07/AutoML_2020_paper_61.pdf.
- Lee DS, McCrary J, Moreira MJ, Porter J (2022). “Valid t -ratio Inference for IV.” *American Economic Review*, **112**(10), 3260–3290. doi:10.1257/aer.20211063.
- Li B, Luo Y, Chai C, Li G, Tang N (2024). “The Dawn of Natural Language to SQL: Are We Fully Ready?” *Proceedings of the VLDB Endowment*, **17**(11), 3318–3331. doi:10.14778/3681954.3682024.
- Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, Wang S, Gan W, Liu Z, Yu Y, Wang Z, Wang Y, Ning W, Hou Y, Wang B, Wu C, Wang X, Liu Y, Wang Y, Tang D, Tu D, Shang L, Jiang X, Tang R, Lian D, Liu Q, Chen E (2025). “ToolACE: Winning the Points of LLM Function Calling.” In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. URL <https://openreview.net/forum?id=8EB8k6DdCU>.
- Longley JW (1967). “An Appraisal of Least Squares Programs for the Electronic Computer from the Point of View of the User.” *Journal of the American Statistical Association*, **62**(319), 819–841. doi:10.1080/01621459.1967.10500896.
- Ludwig J, Mullainathan S, Rambachan A (2024). “Large Language Models: An Applied Econometric Framework.” *NBER Working Paper*, (33344). doi:10.3386/w33344. URL <https://www.nber.org/papers/w33344>.
- McCullough BD (1997). “A Review of **RATS** v4.2: Benchmarking Numerical Accuracy.” *Journal of Applied Econometrics*, **12**(2), 181–190. doi:10.1002/(SICI)1099-1255(199703/04)12:2<181::AID-JAE428>3.0.CO;2-V.
- McCullough BD, Vinod HD (1999). “The Numerical Reliability of Econometric Software.” *Journal of Economic Literature*, **37**(2), 633–665. doi:10.1257/jel.37.2.633.
- Narechania A, Srinivasan A, Stasko J (2021). “NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries.” *IEEE Transactions on Visualization and Computer Graphics*, **27**(2), 369–379. doi:10.1109/TVCG.2020.3030378.
- Ollama Inc (2024). *Ollama: Run Large Language Models Locally*. URL <https://ollama.ai>.
- Pola-rs Contributors (2024). *polars: Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL <https://pola.rs/>.
- Qu C, Dai S, Wei X, Xu J, Cai H, Wang S, Yin D, Wen JR (2025). “Tool Learning with Language Models: A Comprehensive Survey of Methods, Pipelines, and Benchmarks.” *Vicinearth*. doi:10.1007/s44336-025-00024-x.
- Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023). “Toolformer: Language Models Can Teach Themselves to Use Tools.” *arXiv preprint arXiv:2302.04761*. URL <https://arxiv.org/abs/2302.04761>.
- Staiger D, Stock JH (1997). “Instrumental Variables Regression with Weak Instruments.” *Econometrica*, **65**(3), 557–586. doi:10.2307/2171753.

- Stock JH, Yogo M (2005). “Testing for Weak Instruments in Linear IV Regression.” In DWK Andrews, JH Stock (eds.), *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, pp. 80–108. Cambridge University Press, Cambridge. doi:10.1017/CB09780511614491.006.
- Sun L, Abraham S (2021). “Estimating Dynamic Treatment Effects in Event Studies with Heterogeneous Treatment Effects.” *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.
- Sun M, Han R, Jiang B, Qi H, Sun D, Yuan Y, Huang J (2025). “LAMBDA: A Large Model Based Data Agent.” *Journal of the American Statistical Association*. doi:10.1080/01621459.2025.2510000. Online first.
- Thornton C, Hutter F, Hoos HH, Leyton-Brown K (2013). “Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms.” In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 847–855. doi:10.1145/2487575.2487629.
- van der Laan MJ, Rose S (2011). *Targeted Learning: Causal Inference for Observational and Experimental Data*. Springer, New York. doi:10.1007/978-1-4419-9782-1.
- Wilkinson GN, Rogers CE (1973). “Symbolic Description of Factorial Models for Analysis of Variance.” *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, **22**(3), 392–399. doi:10.2307/2346786.
- Woods WA (1973). “Progress in Natural Language Understanding: An Application to Lunar Geology.” In *Proceedings of the AFIPS National Computer Conference*, pp. 441–450. doi:10.1145/1499586.1499695.
- Wooldridge JM (2010). *Econometric Analysis of Cross Section and Panel Data*. 2nd edition. MIT Press, Cambridge, MA.
- Wooldridge JM (2021). “Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators.” *SSRN Working Paper*. doi:10.2139/ssrn.3906345.
- Xu Y (2017). “Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models.” *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.
- Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. URL <https://arxiv.org/abs/2210.03629>.
- Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D (2018). “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.” In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. doi:10.18653/v1/D18-1425.
- Zeileis A (2004). “Econometric Computing with HC and HAC Covariance Matrix Estimators.” *Journal of Statistical Software*, **11**(10), 1–17. doi:10.18637/jss.v011.i10.

Zhong V, Xiong C, Socher R (2017). “Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning.” In *arXiv preprint arXiv:1709.00103*. URL <https://arxiv.org/abs/1709.00103>.

Affiliation:

Ulrich Matter

Bern University of Applied Sciences, Bern, Switzerland

University of St. Gallen, St. Gallen, Switzerland

E-mail: ulrich.matter@unisg.ch

URL: <https://umatter.github.io/>